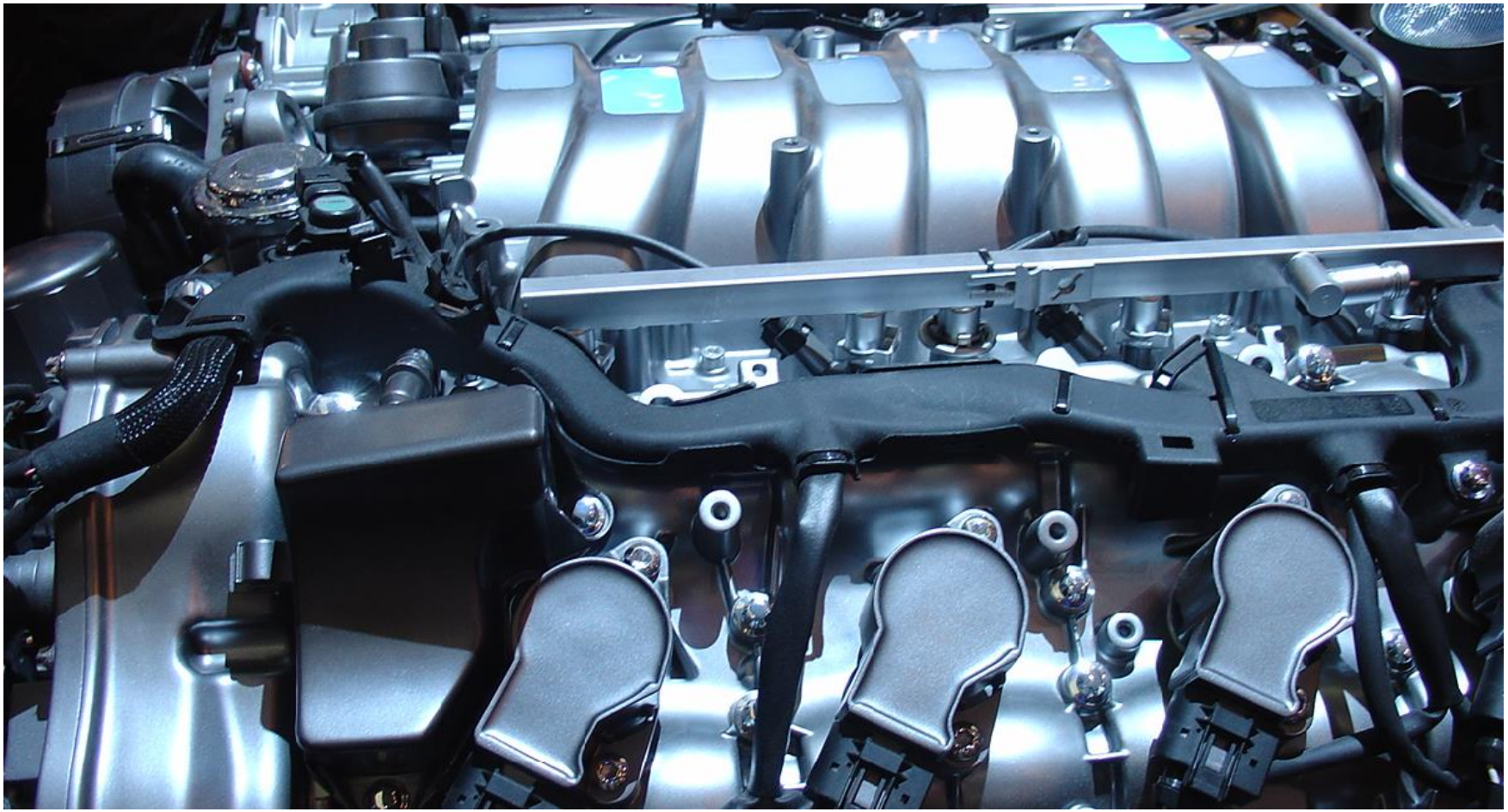


§5 控制系统的软件设计 / Software Design of AECs

- 本章要求/Requirement of this section
  - 汽车电子控制软件的架构/ Architecture of AECs Software
  - 学习教材：汽车软件工程/Learn the Reference: Automotive Software Engineering
  - 了解汽车开发式系统架构的概念和含义/ Understand the Concept of AUTomotive Open System ARchitecture (AUTOSAR)
  - 掌握嵌入式实时系统的编程方法和手段/ Master the development methods of embedded real time system
- 学习参考资料/ Study references:
  - SAE关于软件可靠性标准规范/ SAE software reliability standards;
  - SAE关于故障诊断的标准规范/ SAE diagnosis standards;
  - OSEK/VDX汽车电子嵌入式软件编程技术 北航出版社 Joseph Lemieux等著



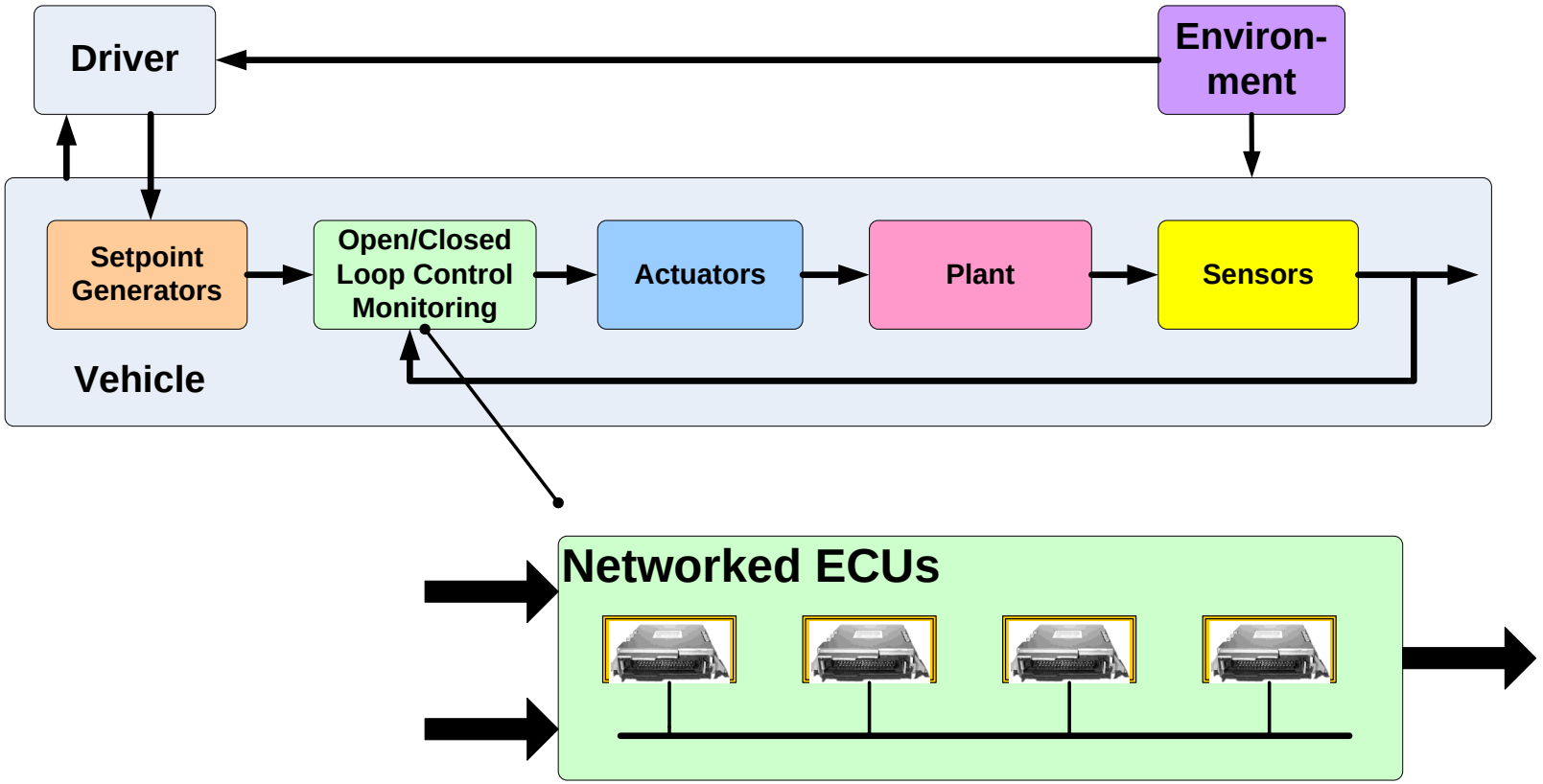
§5.1 汽车电子控制软件的架构/ Architecture of AECs Software





§5.1 汽车电子控制系统的架构
Architecture of AECs

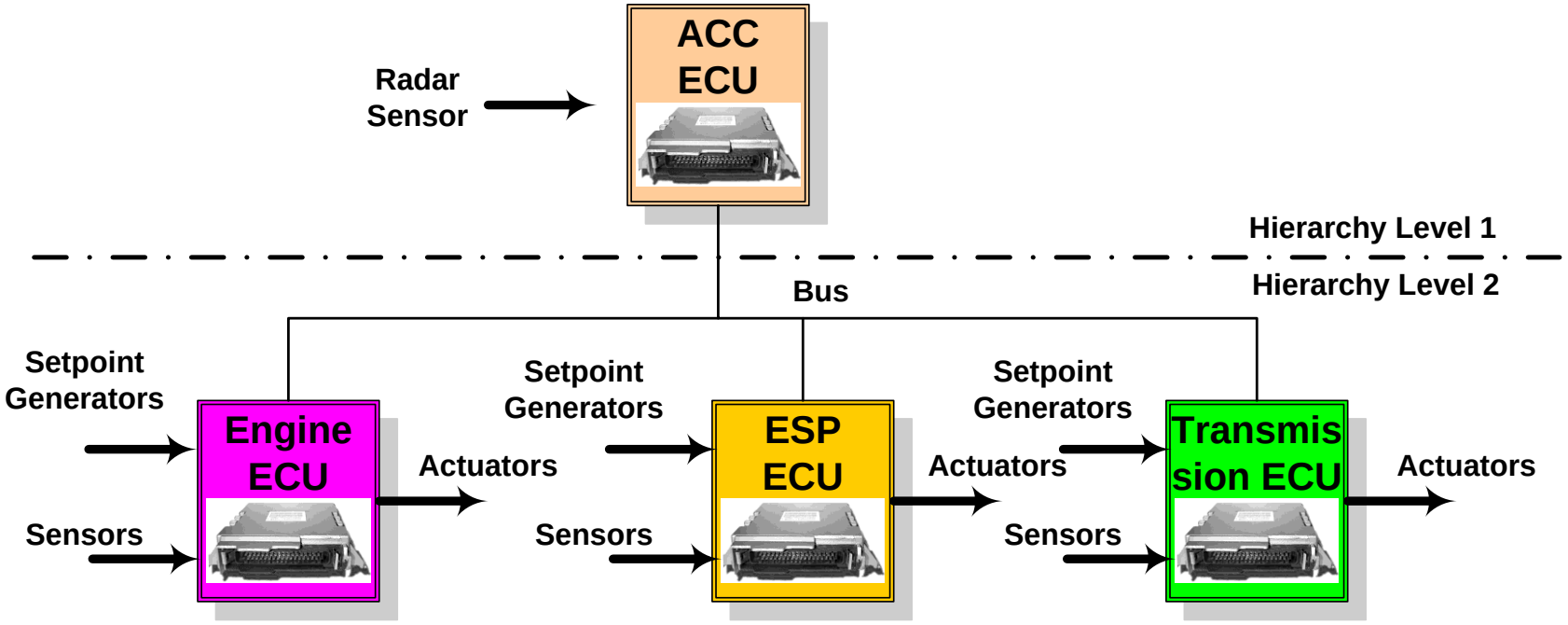
●驾驶员–车辆–环境系统/ The Driver–Vehicle–Environment System





§5.1 汽车电子控制系统的架构
Architecture of AECs

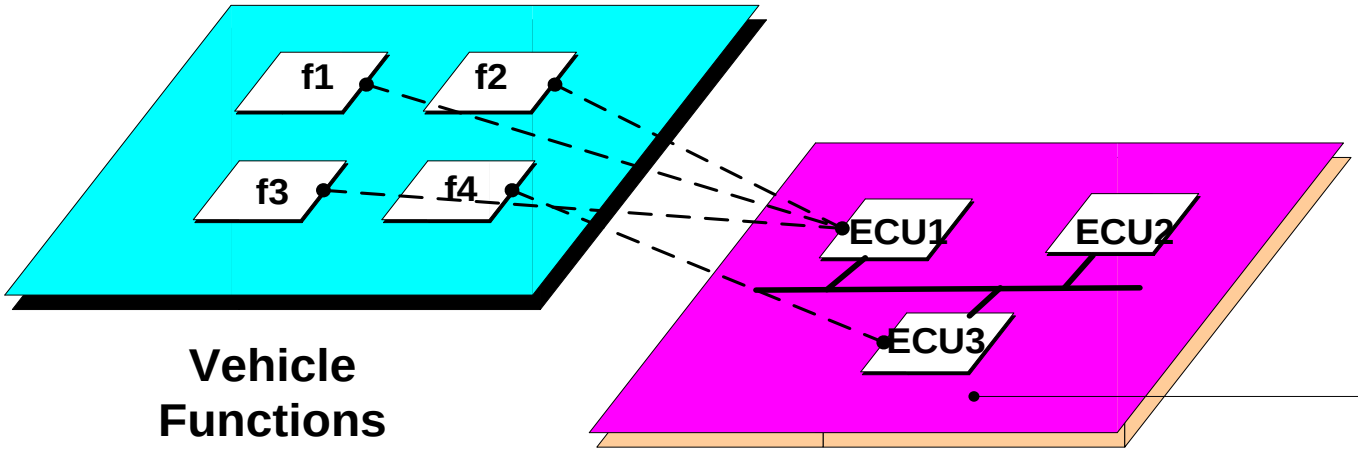
●自适应巡航控制系统/Adaptive Cruise Control System



The ACC ECU and ACC Systems



§5.1 汽车电子控制系统的架构
Architecture of AECs



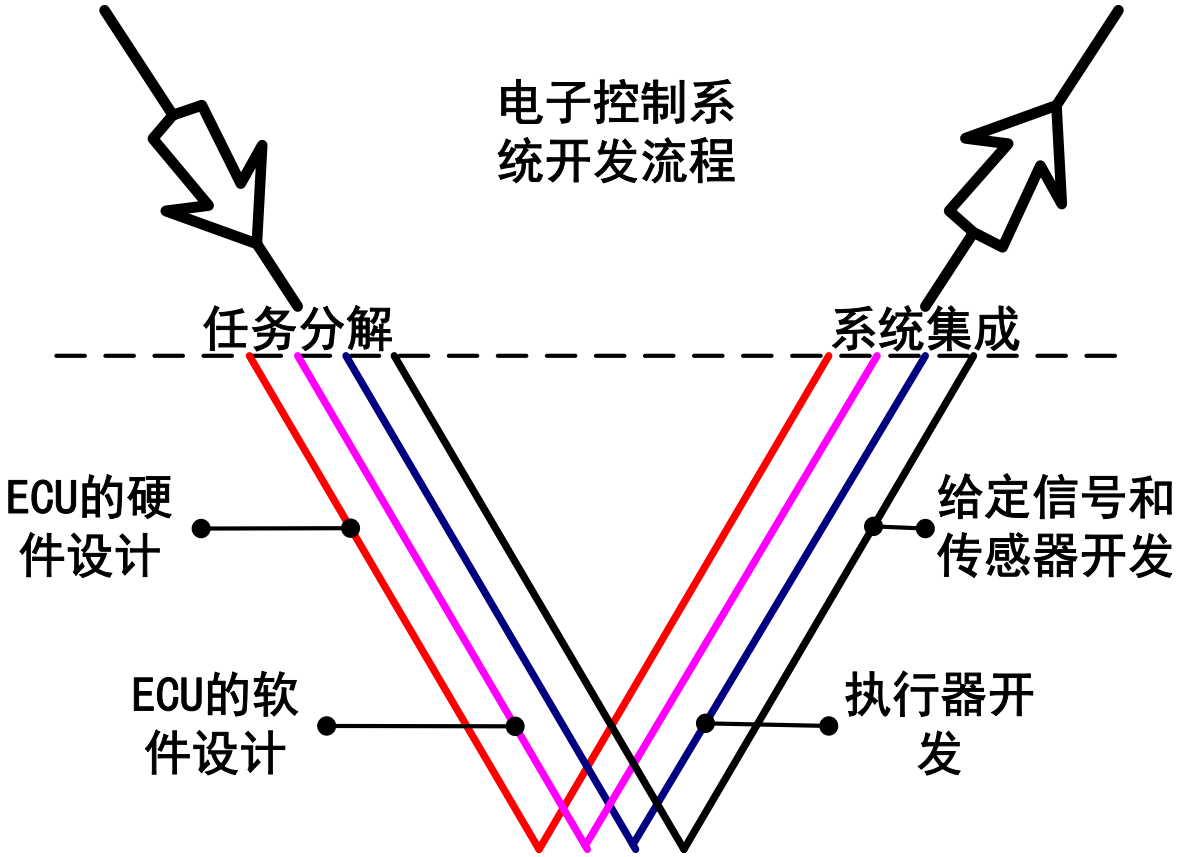


Vehicle

Function and ECU networks of the vehicle

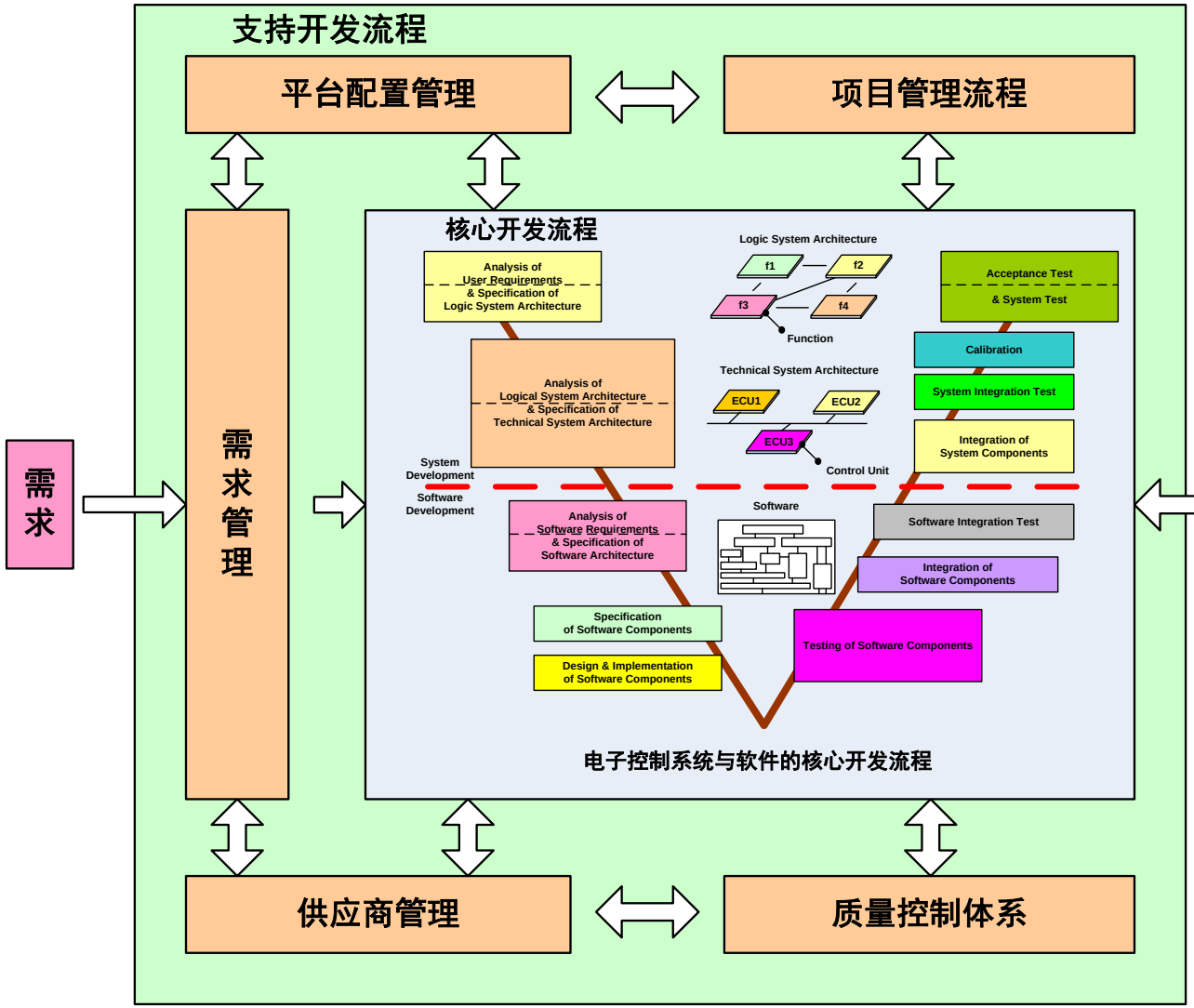


§5.1 汽车电子控制系统的架构
Architecture of AECs





§5.1 汽车电子控制系统的架构
Architecture of AECs

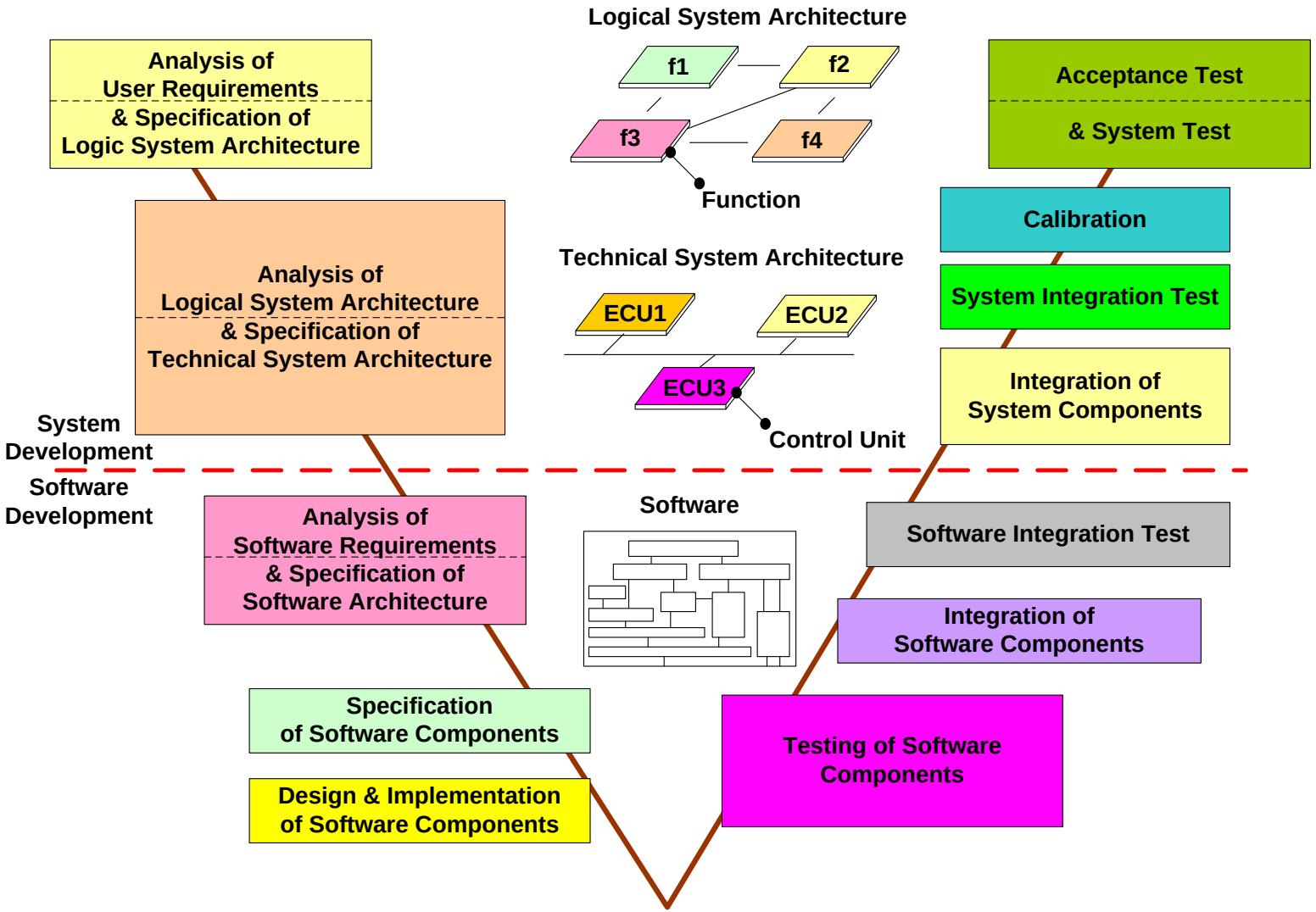




车辆电子控制系统



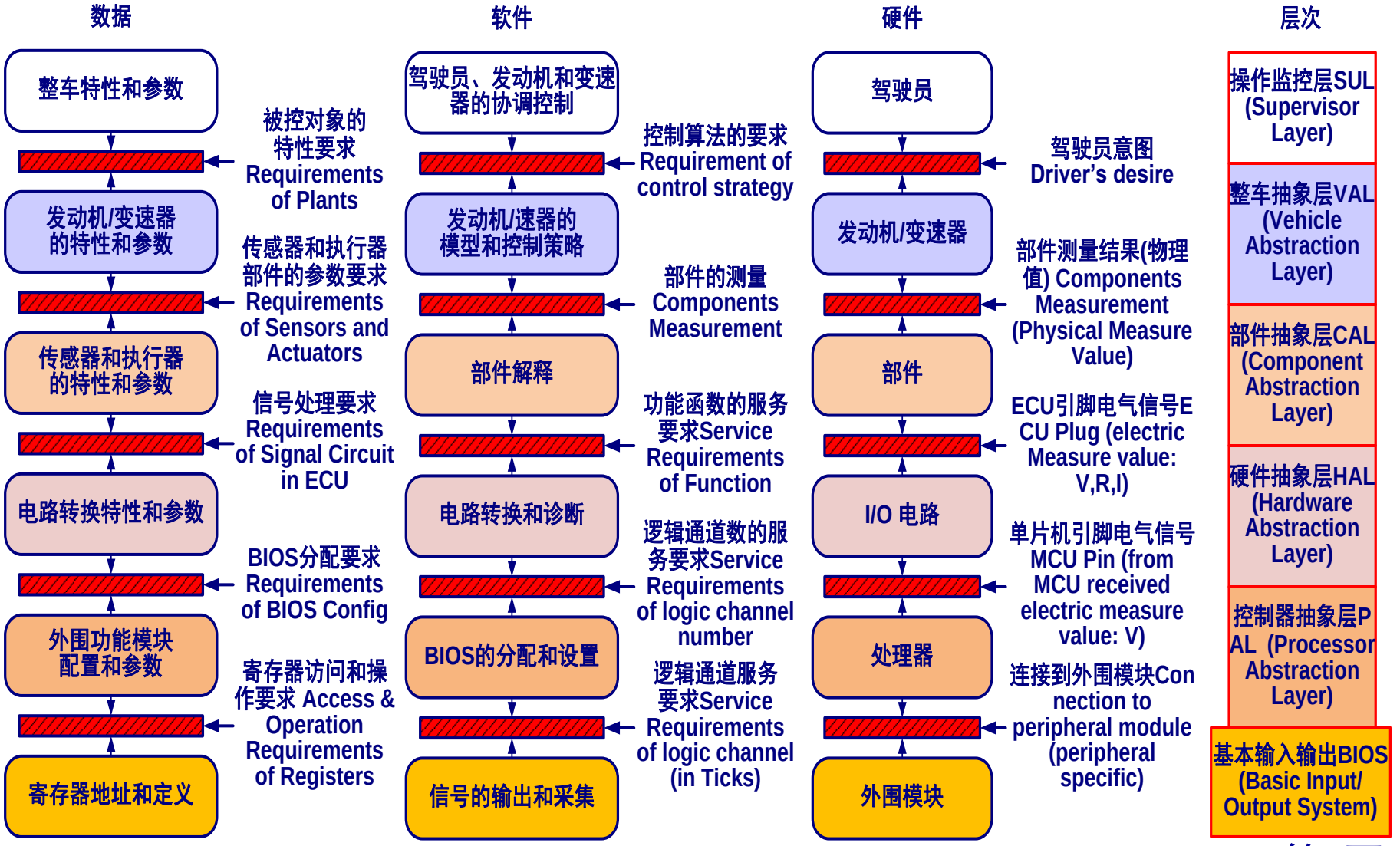
§5.1 汽车电子控制系统的架构
Architecture of AECs





§5.1 汽车电子控制系统的架构
Architecture of AECs

●Software layers





§5.1 汽车电子控制系统的架构
Architecture of AECs

1. Analysis of user requirements and specification of logical system architecture; 在逻辑层次解决客户需求和指标
2. Analysis of the logical system architecture and specification of technical system architecture; 分析逻辑层次的构架，确定技术层次的解决方案；
3. Analysis of software requirements and specification of software architecture; 确定软件部分的解决方案和技术指标
4. Specification of software components; 确定软件模块的指标
5. Design, implementation, and tests of software components；设计和测试软件模块
6. Integration of software components and software integration tests；集成软件模块和软件集成测试
7. Integration of system components and system integration tests；系统集成和测试
8. Calibration；标定
9. System test and acceptance test；系统测试和检验



§5.1 汽车电子控制系统的架构
Architecture of AECs

传统的汽车电子软件设计框架/ Traditional Architecture of AECs Software

- **前后台模式/ Foreground and background Mode**

前/后台系统：一个大循环，循环查询各种标志位。如果标志位置位，就执行相应的服务程序。/ The foreground and background mode is a super loop, which inquiry every flags to implement corresponding service routine.

标志位就是标志事件的发生，事件响应延时处于不可预测状态。/ The Flag presents the events happened. The response delay is un–predicable.

中断服务虽然能即时/优先响应，但是它们和主循环的通讯，也是通过置主循环中相应的标志位来完成的。/The interrupt mechanism provides the priority and real–time to response, but they need the flags to communicate with the main loop.

最坏的情况是循环中所有其他的事件服务程序执行完，才响应当前事件。/ In the worst case, the response current event can only happened after all the other events have been implemented.



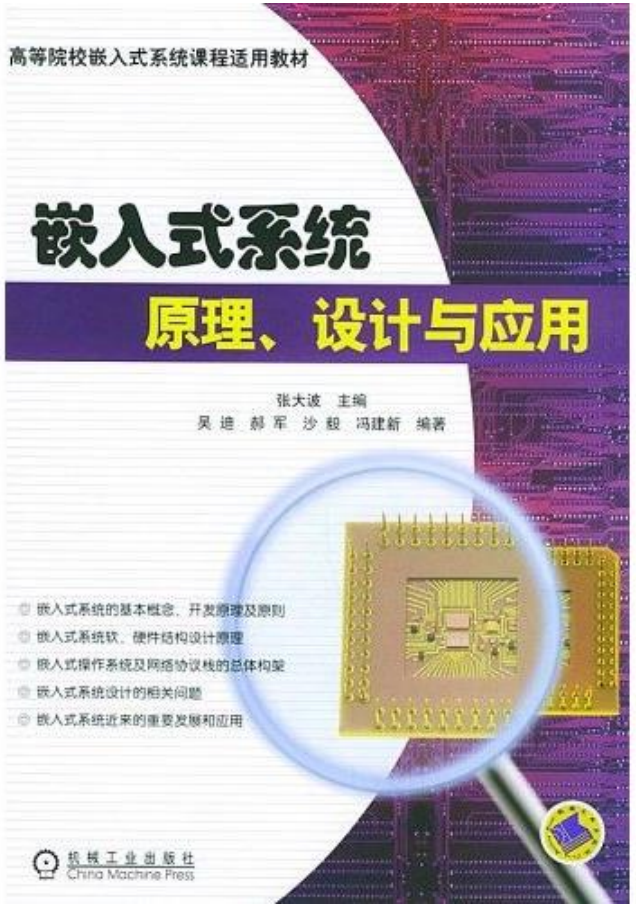
传统的汽车电子软件设计框架/ Traditional Architecture of AECs Software

- 什么是后台，什么是前台?

后台：应用程序是一个无限循环，巡回地执行多个事件，完成相应的操作，这一部分软件称为后台，通常在主程序main()中被调用。/ Background is an infinite loop, generally is called by main.

前台：一般由中断服务程序组成，中断服务程序处理异步事件，这一部分可以看成是前台。/Foreground generally consists of interrupt service routine.

因此，后台也可称为任务级，而前台则被称为中断级。/Hence background is tasks and Foreground is interrupts.





§5.1 汽车电子控制系统的架构
Architecture of AECs

传统的汽车电子软件设计框架/ Traditional Architecture of AECs Software

- 前后台设计

```
main()
{
   /* 硬件初始化 */

   /* 后台程序 */
   while (1)
   {
      action1();
      action2();
      ...
      actionn();
   }
}
```

```
action1()
{   /* 执行操作1 */
    ...
}
action2()
{   /* 执行操作2 */
    ...
}
    ...
actionn()
{   /* 执行操作n */
    ...
}
```

后台

```
isr1()
{   /* 中断1的中断服务程序 */
    ...
}
isr2()
{   /* 中断2的中断服务程序 */
    ...
}
   ...
isrn()
{   /* 中断n的中断服务程序 */
    ...
}
```

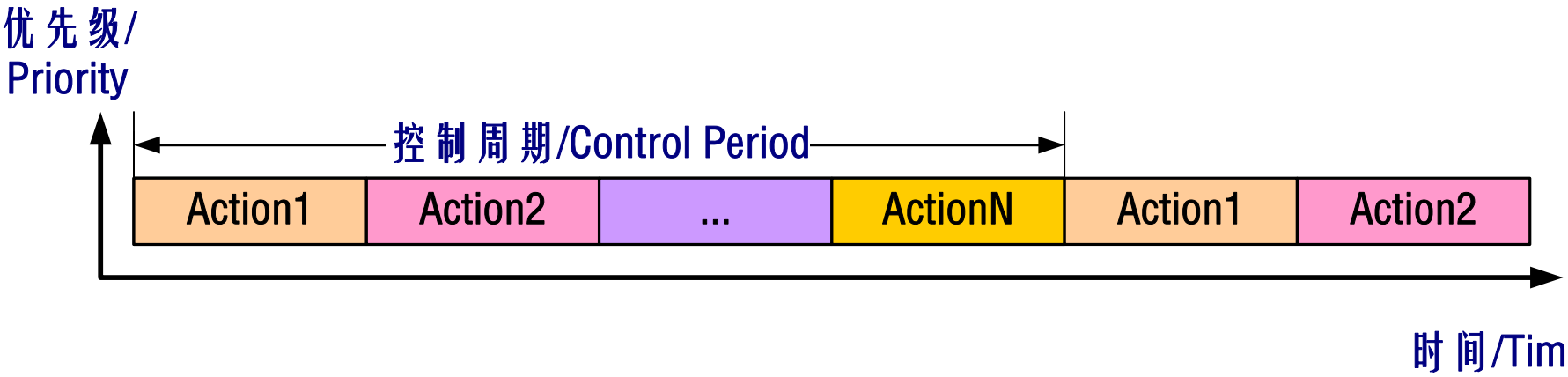
前台



§5.1 汽车电子控制系统的架构
Architecture of AECs

传统的汽车电子软件设计框架/ Traditional Architecture of AECs Software

- 前后台设计



- 巡回服务系统中，处理器总是处于全速运行状态（即我们通常所说的Poll轮循方法），处理器的开销比较大，带来的问题是能耗高。如果系统的外部事件不是发生得很频繁，那么可以降低处理器服务事件的频率，同时这样也不会降低响应时间，这时就采用定时器驱动的巡回服务系统。
- 同时这种巡回服务系统，控制周期不是很精确，为保证控制周期的精确度，例如标准的10ms控制周期，往往采用定时器触发，每隔10ms产生一次定时器中断，一般又叫Period Interrupt Timer，简称PIT。



§5.1 汽车电子控制系统的架构
Architecture of AECs

传统的汽车电子软件设计框架/ Traditional Architecture of AECs Software

●采用定时器触发的后台设计，其主程序只有初始化函数，任务调度放在定时器中断服务程序中。

```
main()
{
   /* 系统初始化 */

   /* 设置定时器 */

   while (1)

enter_low_power();
}
```

```
isr_timer() /* 定时器的中断处理程序 */
{
   action1(); /* 执行事件1的处理 */
   action2(); /* 执行事件2的处理 */
   ...
   actionn(); /* 执行事件n的处理 */
}
```

后台



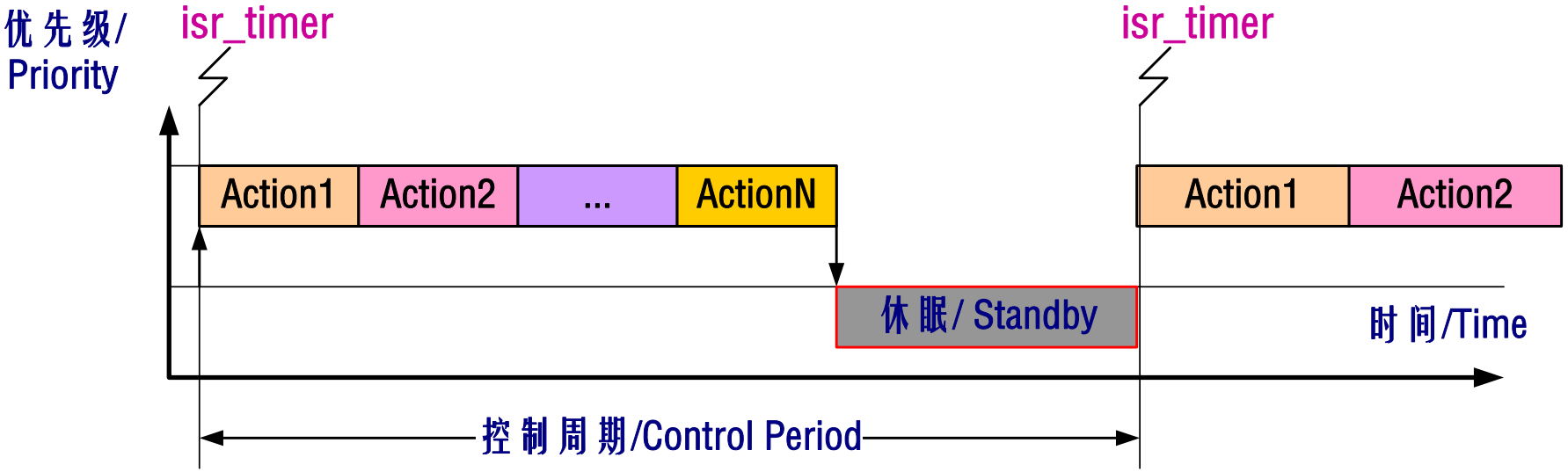
§5.1 汽车电子控制系统的架构
Architecture of AECs

传统的汽车电子软件设计框架/ Traditional Architecture of AECs Software

- 前后台设计

采用定时器配置的后台配置/ The background of triggered by timer.

- 控制周期可以由定时精确控制；/ Period is precisely controlled by timer.
- 没有任务时可以进入睡眠模式；/ CPU enter standby mode without task.

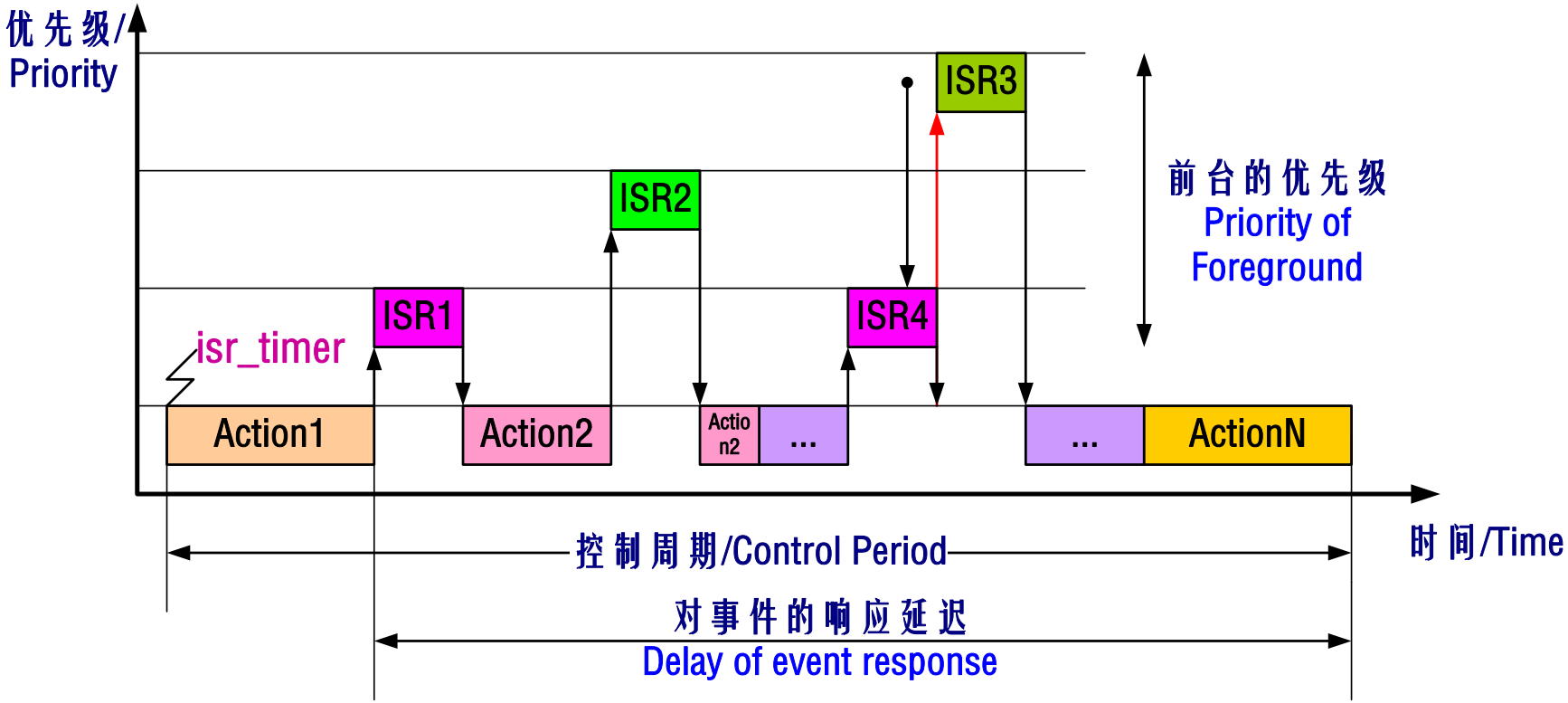




§5.1 汽车电子控制系统的架构
Architecture of AECs

传统的汽车电子软件设计框架/ Traditional Architecture of AECs Software

●前台的设计，就转变为中断的配置，通常在定时器中断服务程序中要重新开中断允许，并且中断的优先级要高于定时器中断ISR_timer，这样前台的程序ISR1~ISR3等就可以得到及时的响应。





§5.1 汽车电子控制系统的架构
Architecture of AECs

传统的汽车电子软件设计框架/ Traditional Architecture of AECs Software

- 前后台举例：转速信号测量(实验二)

```
/*后台模板*/

main()
{
  /* 硬件初始化 */

  /* 后台程序 */
  while (1)
  {
     action1();
     action2();
     ...
     actionn();
  }
}
```

```
void main(void)
{
  /*** Processor Expert intern
REMOVE THIS CODE!!! ***/
  PE_low_level_init();
  /*** End of Processor Exper
***/
  /* Write your code here */
  Cap1_Enable();
  /*** Processor Expert end o
MODIFY THIS CODE!!! ***/
  for(;;){}
  /*** Processor Expert end o
WRITE CODE BELOW!!! ***/
} /*** End of main routine. DO
***/
```

```
byte Cap1_Enable(void)
{
  T1SC_TSTOP = 0;
/* Run counter */
   /* T1SC0:
CH0F=0,CH0IE=1,MS0B=0,MS0
A=0,ELS0B=0,ELS0A=1,TOV0=
0,CH0MAX=0 */
  T1SC0 = 0x44;
/* Enable both interrupt and
capture function */
  return ERR_OK;
/* OK */
}
```



传统的汽车电子软件设计框架/ Traditional Architecture of AECs Software

- 前后台举例：转速信号测量(实验二)

```
volatile unsigned int Capturevalue0;
volatile unsigned int CaptureDiff;
volatile unsigned int ;
void Cap1_OnCapture(void)
{
  static unsigned char CaptureFlag = 1;
  if(CaptureFlag == 1) {
    Capturevalue0 = Cap1_GetCaptureValue();
  } else {
    CaptureDiff = Capturevalue0 – Cap1_GetCaptureValue();
  }
  CaptureFlag  =  1 – CaptureFlag ;
  Cap1_Enable();
/* Write your code here ... */
}
/* END Events */
```

```
ISR(Cap1_Interrupt)
{
  T1SC0_CH0F = 0;
   /* Reset interrupt request flag */
  Cap1_OnCapture();
    /* Invoke user event */
}

/* END Cap1. */
```



现代的汽车电子软件设计框架/ Traditional Architecture of AECs Software

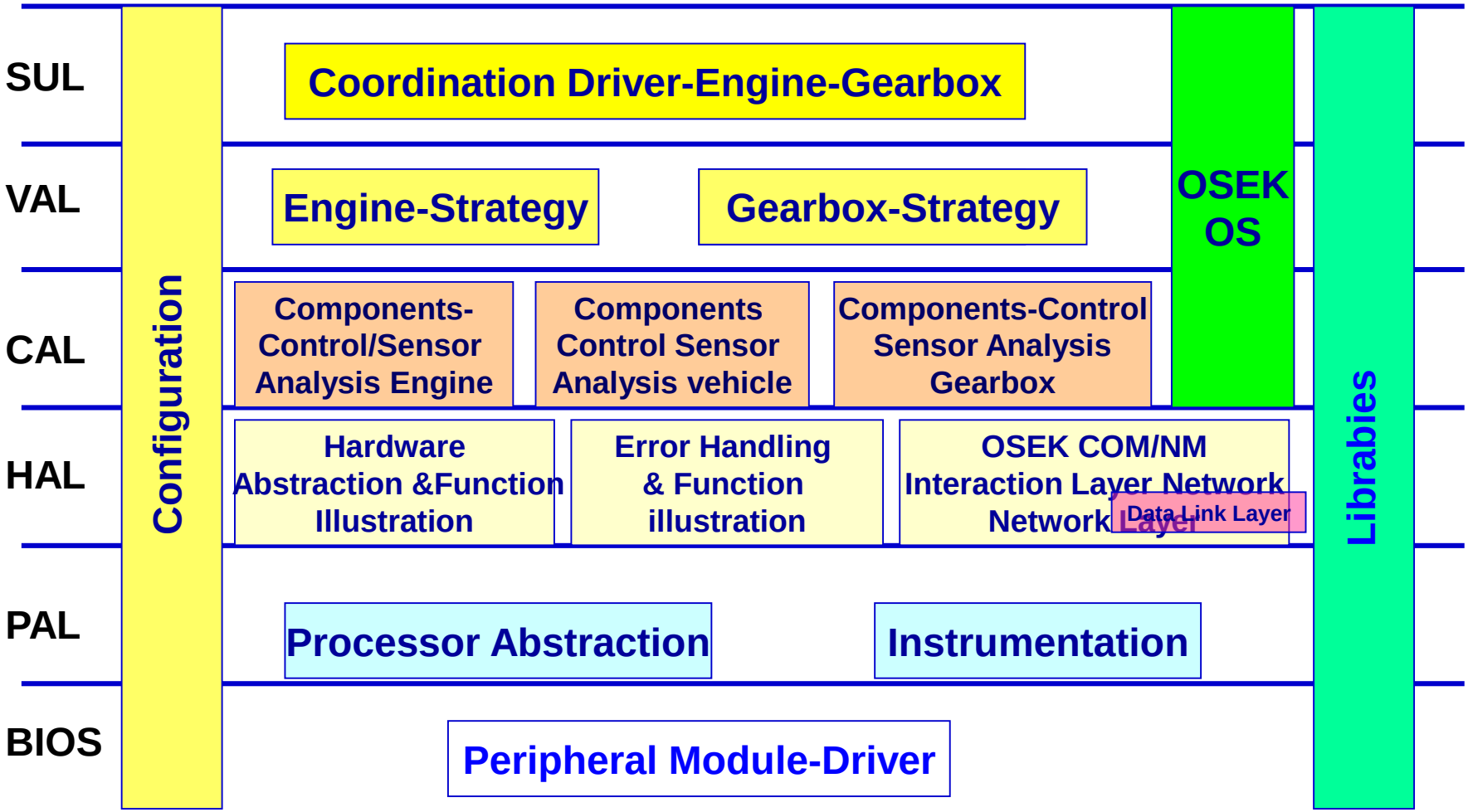
- 采用嵌入式实时操作系统的模式/ Embedded Real-Time Operating System
  - 实时系统（uCOS）：整个程序分成一个个看起来好象是并行的任务，每个任务都在等待事件的发生。因为uCOS不支持时间片轮转，除了最低优先级任务（在uCOS中是IDLE任务）是死循环以外，其他的任务都不能死循环，只能在驱动事件驱动下工作。任何驱动事件的产生，都使优先级最高的就绪任务运行。任务和任务/任务和中断的通讯，是通过相应事件驱动来完成的。

链接：OSEK嵌入式实时操作系统介绍



§5.1 汽车电子控制系统的架构
Architecture of AECs

●OSEK /VDX





§5.1 汽车电子控制系统的架构
Architecture of AECs

- OSEK原文来自德语“Offene Systeme und deren Schnittstellen für die Elektronik im Kraftfahrzeug”的缩写，
- 英文的意思为“Open systems and the corresponding interfaces for automotive electronics”，
- 中文可译为“汽车电子开放式系统及其接口”。
- OSEK最初由欧洲的汽车制造商于1994年提出，后来逐步发展为汽车工业的全球标准。
- OSEK主要包括了四个部分：
  - OSEK OS，操作系统
  - OSEK OIL，可执行语言
  - OSEK COM，通信管理
  - OSEK NM，网络管理



§5.1 汽车电子控制系统的架构
Architecture of AECs

- **操作系统（OS）**
  - ◆包括操作系统功能元素的描述和定义，例如任务（Tasks）、中断（Interrupts）、事件（Events）、计数器（Counters）、警报（Alarms）、资源（Resources）和钩子（Hooks）等基本要素；
  - ◆以及如何定义应用程序接口（Application Programming Interface，简称API），例如功能定义（Functions），类型定义（Type definitions），常数变量定义（Constants）等；
  - ◆还包括进程调度（Scheduling）和优先级（Priority）如何确定和优化的方法；通过操作系统的调度，可以很灵活地适应汽车控制系统中的各种任务之间的嵌套。
- **典型的OSEK OS系统：**
  - ◆Winder River公司的OSEKWorks操作系统；
  - ◆ETAS公司的RTA-OSEK；
  - ◆Freescale公司的OSEK-Turbo；

  上述操作系统均满足OSEK的指标规范（Specification），而且覆盖了汽车电子控制系统中广泛采用了各种8位、16位和32位单片机型号。



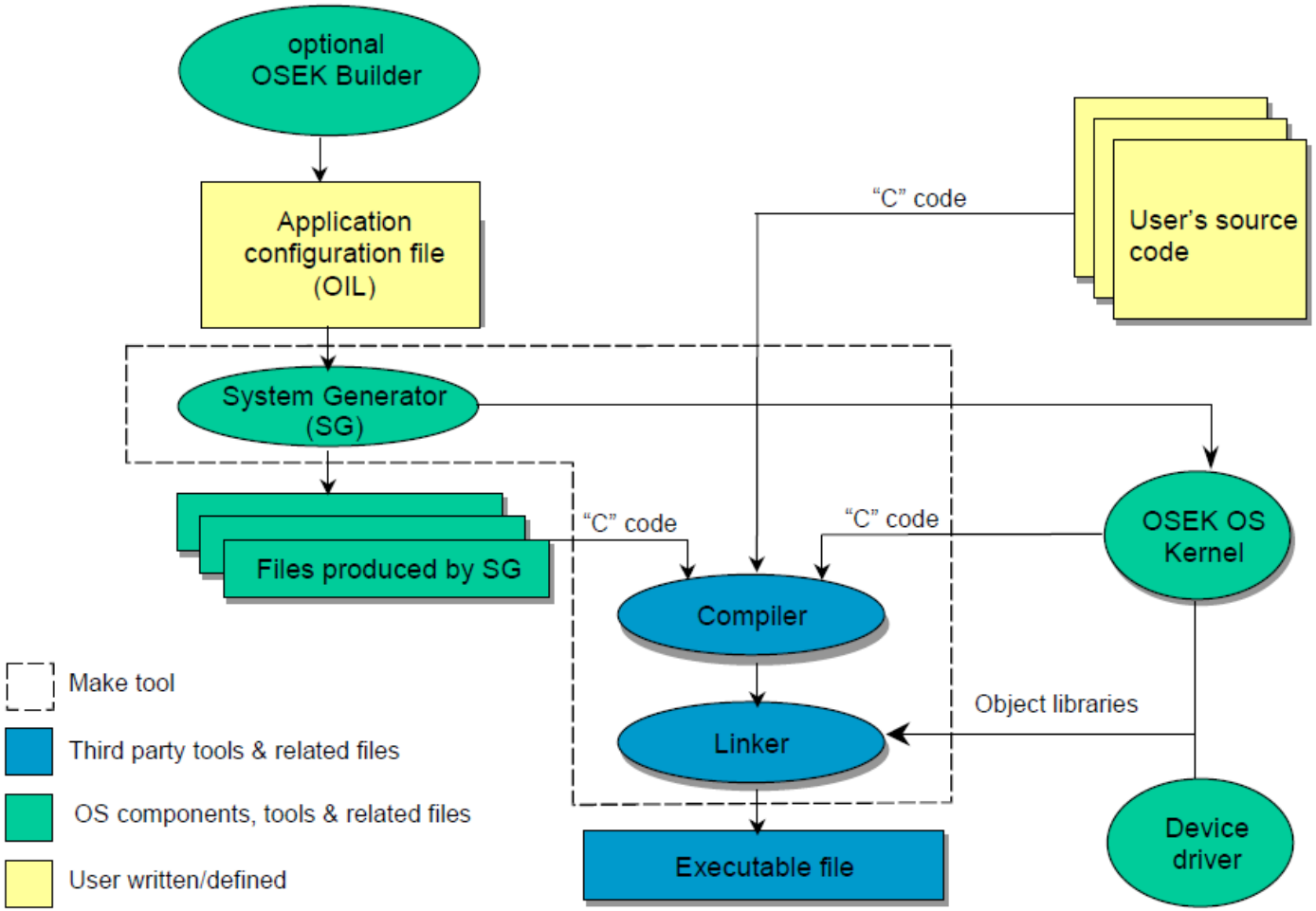
§5.1 汽车电子控制系统的架构
Architecture of AECs

- 可执行语言(OSEK Implementation Language，简称OIL)
- 它在OSEK OS的基础上，帮助用户如何具体定义自己的应用（Application）。只要通过OIL来静态地配置各个任务属性、占有的资源、定义事件等工作，就能够确定程序跑起来之后，操作系统就能够实现动态调度任务的功能。
- 也就是操作系统本身，包括调度的机制、中断嵌套、底层驱动都已经写好，只要解决了操作系统的应用配置（Application Configuration）就可以让操作系统进行任务管理了。通过OIL的配置文件，实现用户对OS的缩放（Scaling）和裁剪，从而使得操作系统能够适应各种各样的汽车电子控制系统的要求。
- OIL代表了用户对操作系统的配置文件，OSEK的创建工具（Make Tool）可将其通过System Generator转化为C文件，结合OSEK OS的C文件与用户自己编程的C文件，一起由单片机的编辑器（例如Codewarrior等）编译、连接成可执行代码文件。



§5.1 汽车电子控制系统的架构
Architecture of AECs

- 可执行语言(OSEK Implementation Language，简称OIL)





§5.1 汽车电子控制系统的架构
Architecture of AECs

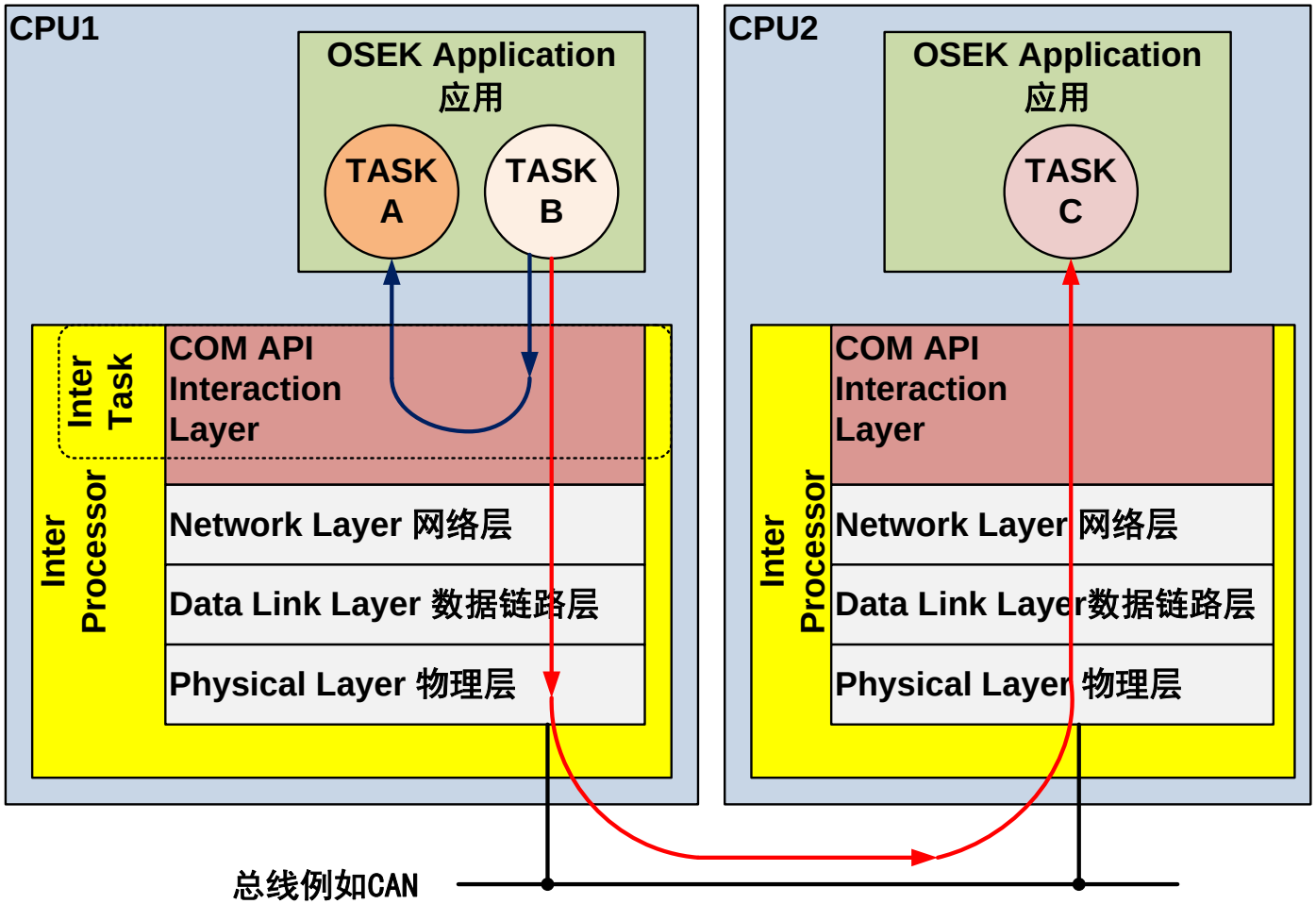
- **通信管理（COM）**
- 主要负责在任务之间的数据通信服务或者通过中断服务进行的通信服务，不同的任务通信可以发生在同一个ECU内部，也可以是多个ECU之间的外部通信服务。
- 通信服务只能通过特殊的应用程序接口（Application Program Interface，简称API）进行接口；
- OSEK COM的目标是为了使得应用软件具有良好的开放接口（Portability）、良好的复用性能（Reusability）和良好的协调操作的性能（Interoperability）。
- 而且通过API能够将内部和外部通信之间的差别，以及不同的底层通讯协议、总线类型和网络的差别隐藏起来，使得上层的应用（Application）独立于底层的通信。

这个思路对后面介绍的AUTOSAR也是适用的！



§5.1 汽车电子控制系统的架构
Architecture of AECs

●通信管理（COM）





§5.1 汽车电子控制系统的架构
Architecture of AECs

●网络管理（NM）

●网络管理的基本目标是为了保证ECU之间的通信的安全性和可靠性，网络管理具有下述的几个功能：

- ◆OSEK各个层次之间的接口，例如与API接口；
- ◆OSEK的内部NM和COM之间的交互接口等；
- ◆适应不同总线协议的要求，例如CAN、LIN、J1850，K-Bus等协议，以及相应的错误处理、状态统计算法等；
- ◆适应不同的节点资源，例如功能根据节点的要求对NM进行缩放，或者根据应用对网络管理服务进行定制等；
- ◆还能够适应硬件设计需求，例如适应协议电路或者物理层电路的特性等；
- ◆此外网络管理还能够适应非OSEK规范的网络通信管理（Station Management）和节点的管理策略等。



§5.1 汽车电子控制系统的架构
Architecture of AECs

●网络管理（NM）

1）API（应用程序接口）；

2）连接到微控制器的各种总线接口；

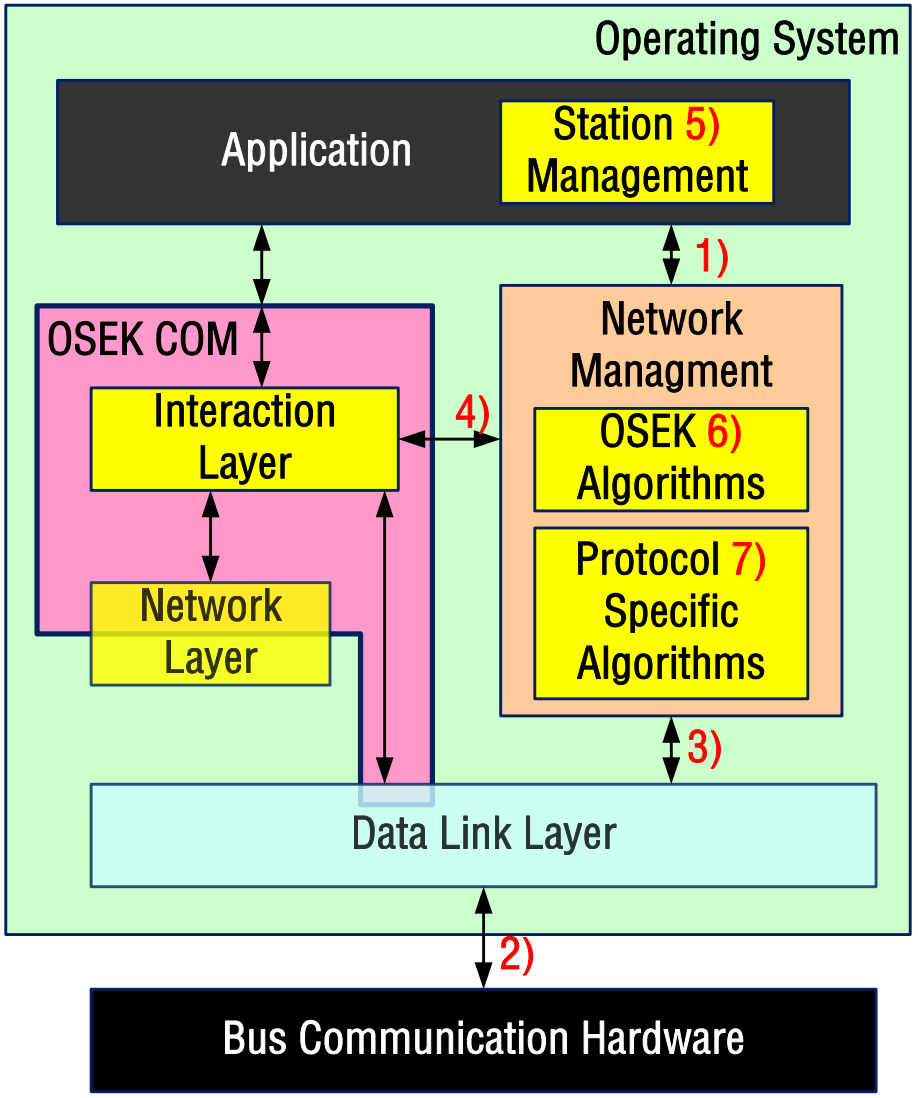
3）和动态链接库（DLL）的接口；

4）和通信管理COM接口；

5）非OSEK标准类型的通信站点管理算法；

6）OSEK管理算法；

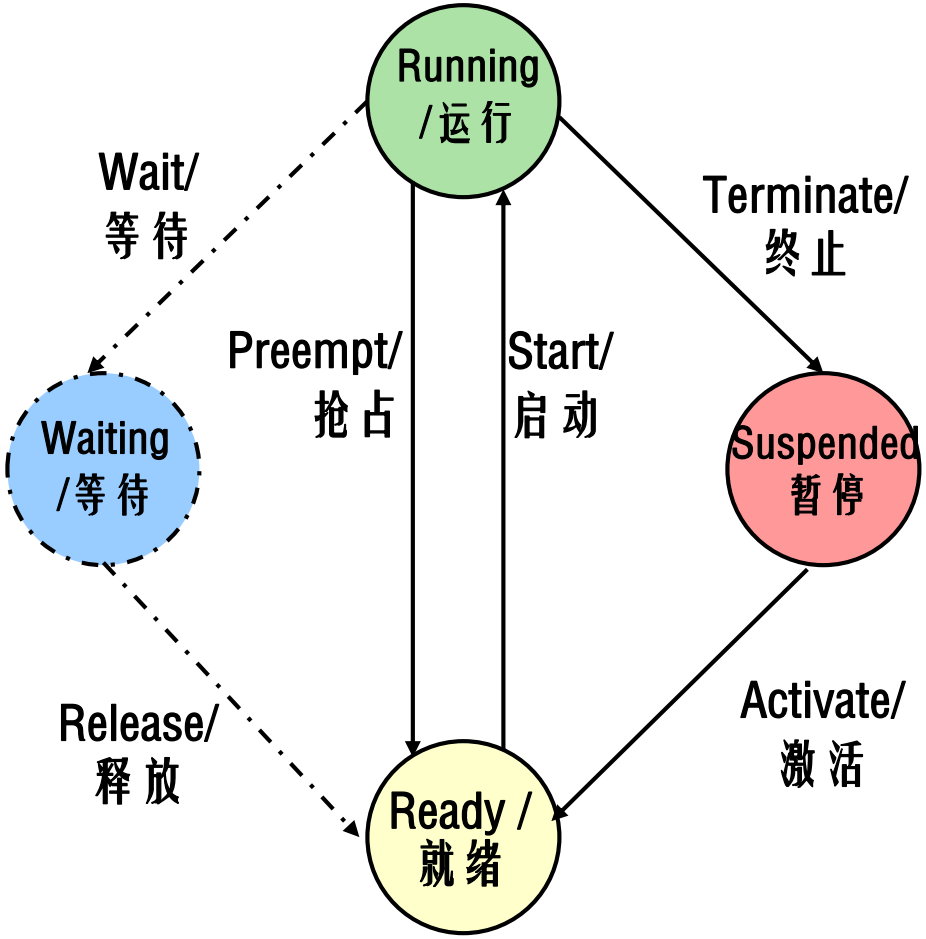
7）协议制定的网络管理算法





●OSEK问题1：任务状态如何标志?

- Running： In the *running* state, the CPU is assigned to the task；
- Ready: All functional prerequisites for a transition into the running state exist, and the task only waits for allocation of the processor.
- Waiting: A task cannot continue execution because it shall wait for at least one event.
- Suspended: In the suspended state the task is passive and can be activated.
- 扩展任务才有Waiting，基本任务只有另外三种状态!



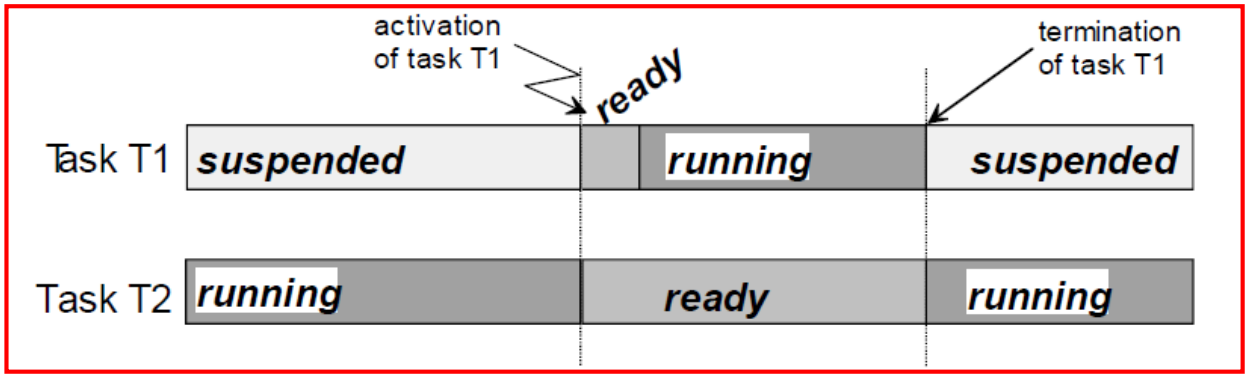


●OSEK问题2：任务是如何调度的?

（1）Full preemptive scheduling/完全先占式调度模式

在下图中，当优先级比较低的Task T2正在运行时，Task T1被一个事件激活，于是T1立刻由暂停（挂起，Suspended）状态转入就绪（Ready）状态，T2立刻由Running转入Ready状态。然后处理器响应T1，即T1进入运行状态，而T2处于就绪状态；当T1终止后，处理器继续运行T2状态。

任务T1实时性重要得多，可随时打断别的任务!

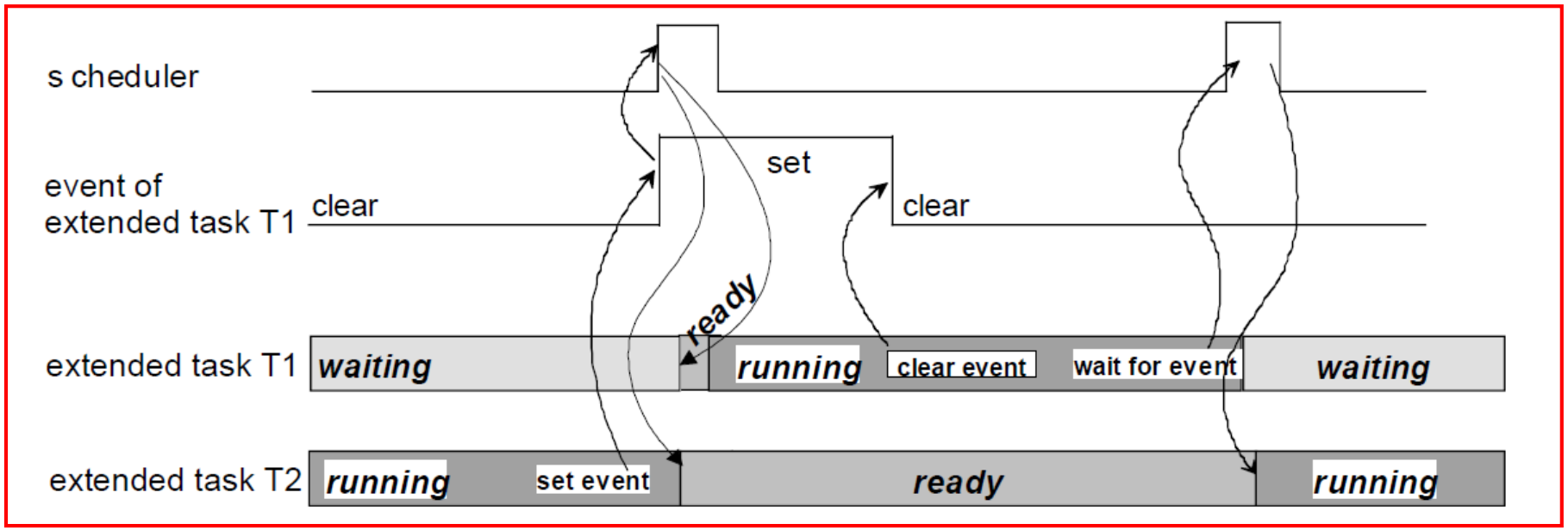




●OSEK问题2：任务是如何调度的?

（1）Full preemptive scheduling/完全先占式调度模式

用事件触发扩展任务的同步方式：



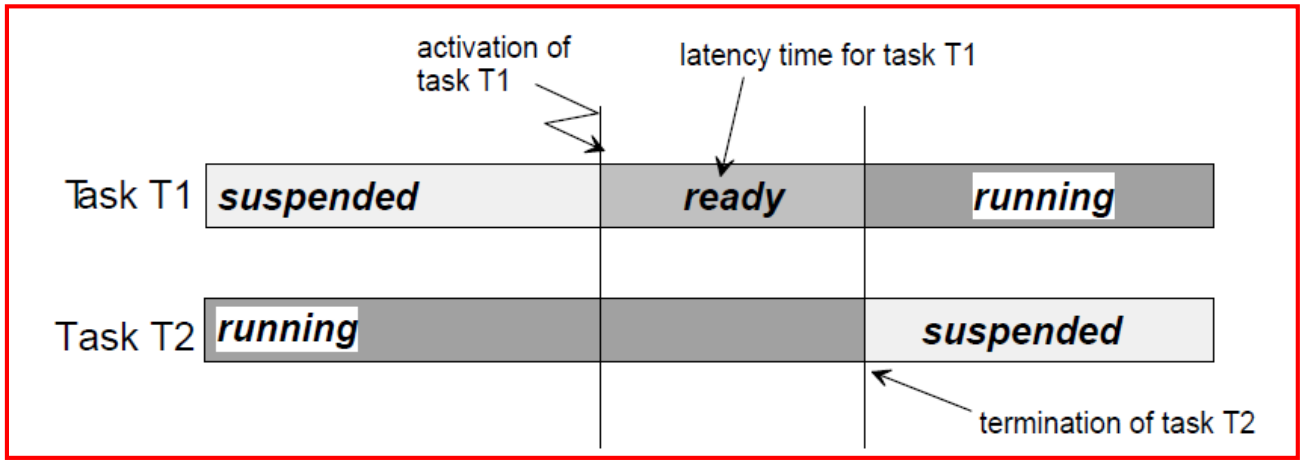


●OSEK问题2：任务是如何调度的?

（2）Non preemptive scheduling/非先占式调度模式

在下图中，当优先级比较低的Task T2正在运行时，Task T1被一个事件激活，于是T1立刻由暂停（挂起，Suspended）状态转入就绪（Ready）状态，但是由于T2任务还没有结束，这时处理器继续执行T2，当T2终止后，处理器才能把T1转入运行状态。

就算任务T1来了，我也执行完任务T2先！

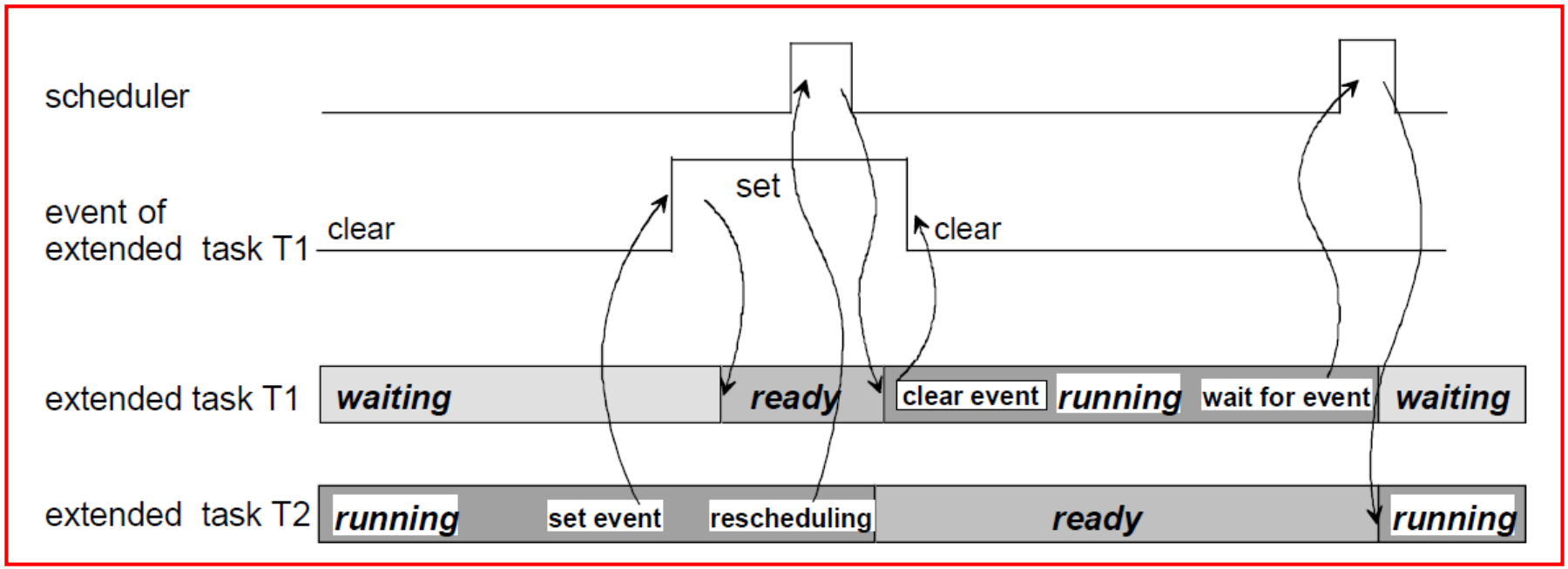




●OSEK问题2：任务是如何调度的?

（2）Non preemptive scheduling/非先占式调度模式

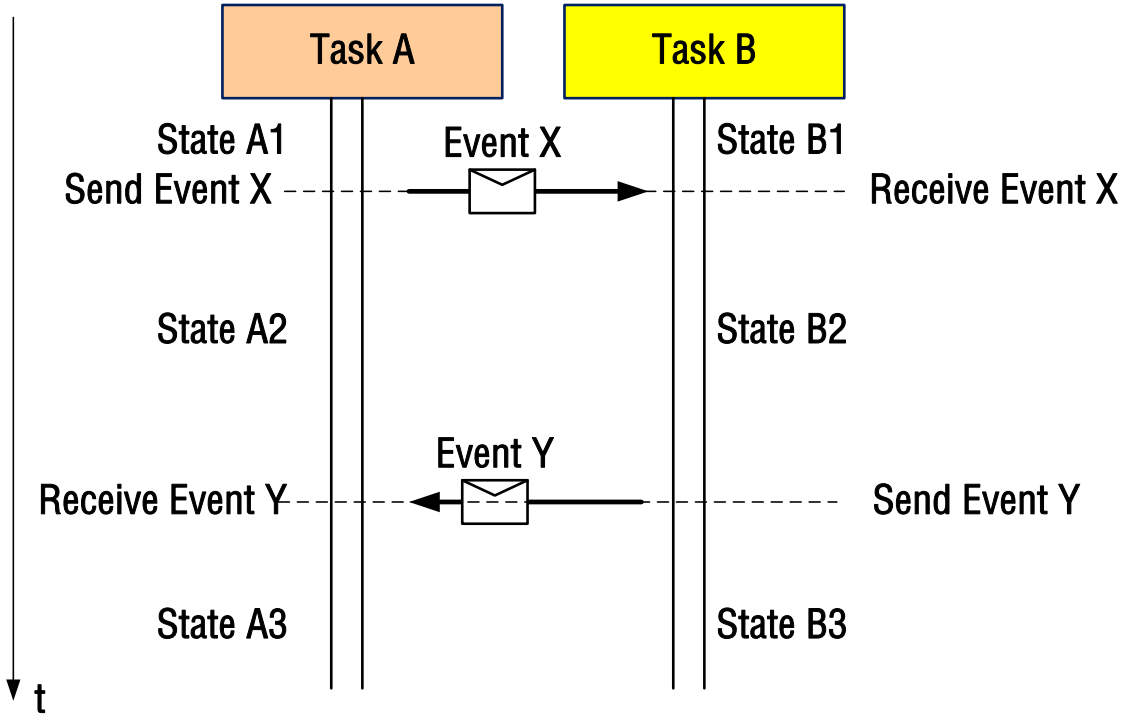
用事件触发扩展任务的同步方式：

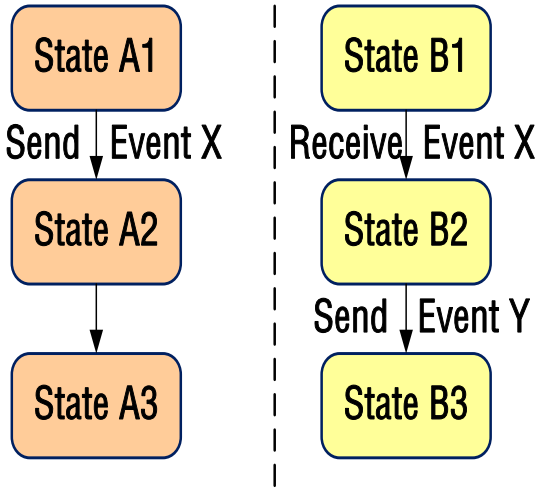




同步/Synchronization

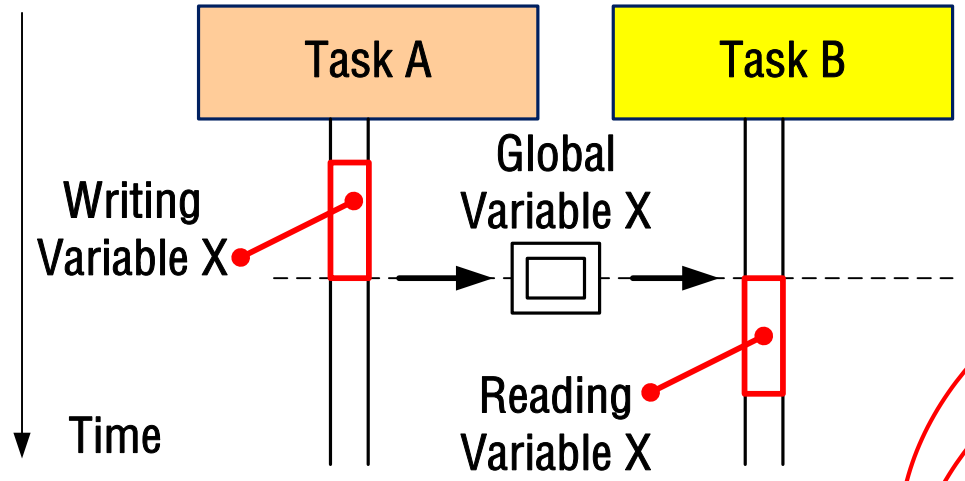
事件的作用：在下例中，任务A和B通过事件X和Y实现状态同步







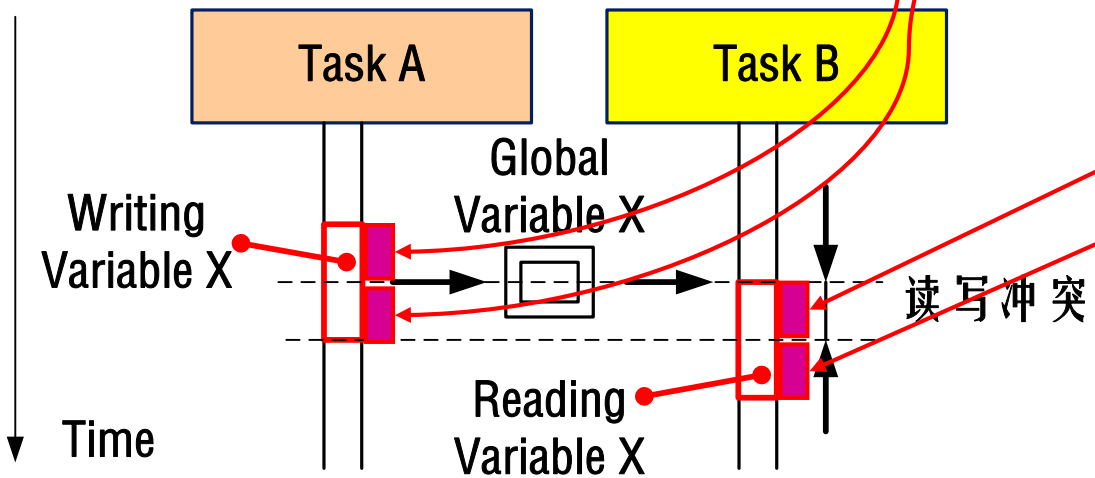
协同/Cooperation



例如X为2字节变量，
X的初值=0xC066；
Task A中给X赋值，
赋值指令为：

```
X_High_Byte=0x80;
X_Low_Byte=0x33;
```

B中断



Task B中读取X的值，
读取指令为：

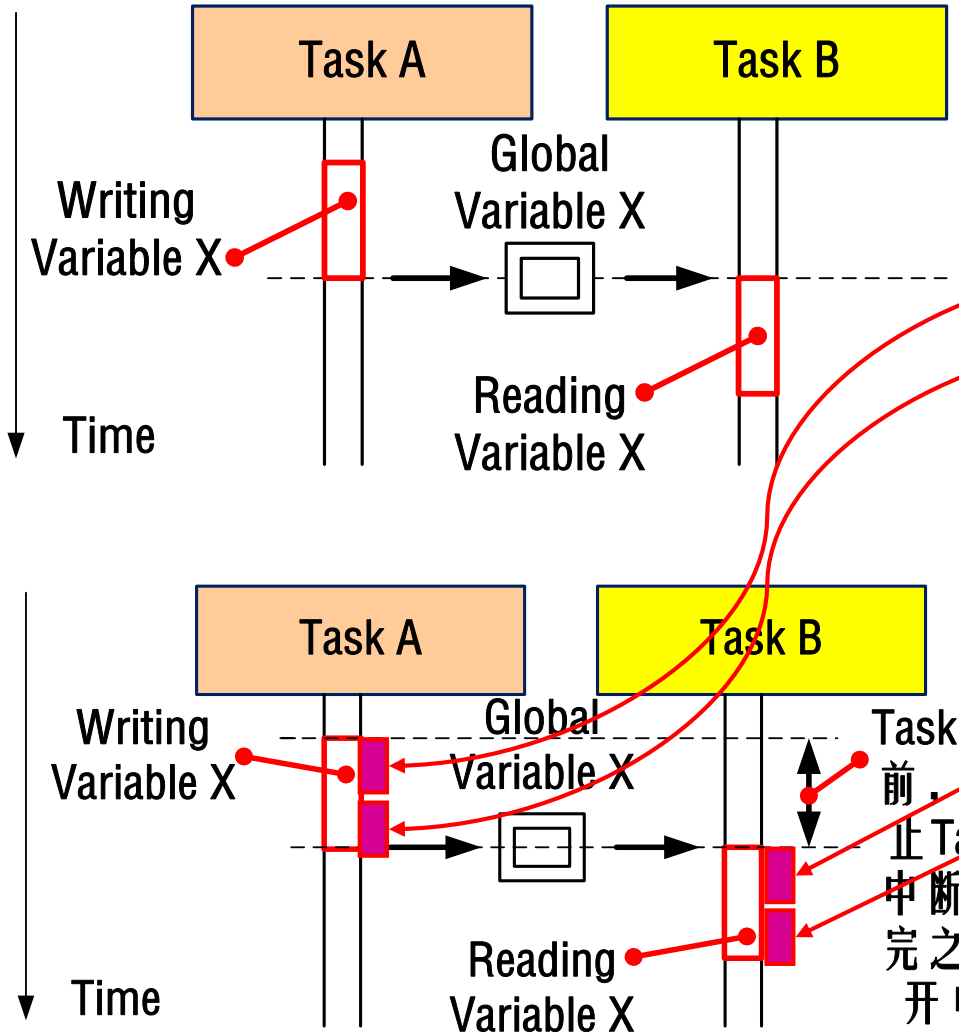
```
Temp1=X_High_Byte;
Temp2=X_Low_Byte;
X= Temp1*256+Temp2;
```

Task B读的结果为：
X=0x8066



§5.1 汽车电子控制系统的架构
Architecture of AECs

协同/Cooperation



例如X为2字节变量，
X的初值=0xC066；
Task A中给X赋值，
赋值指令为：

```
X_High_Byte=0x80;
X_Low_Byte=0x33;
```

禁止中断

Task B中读取X的值，
读取指令为：

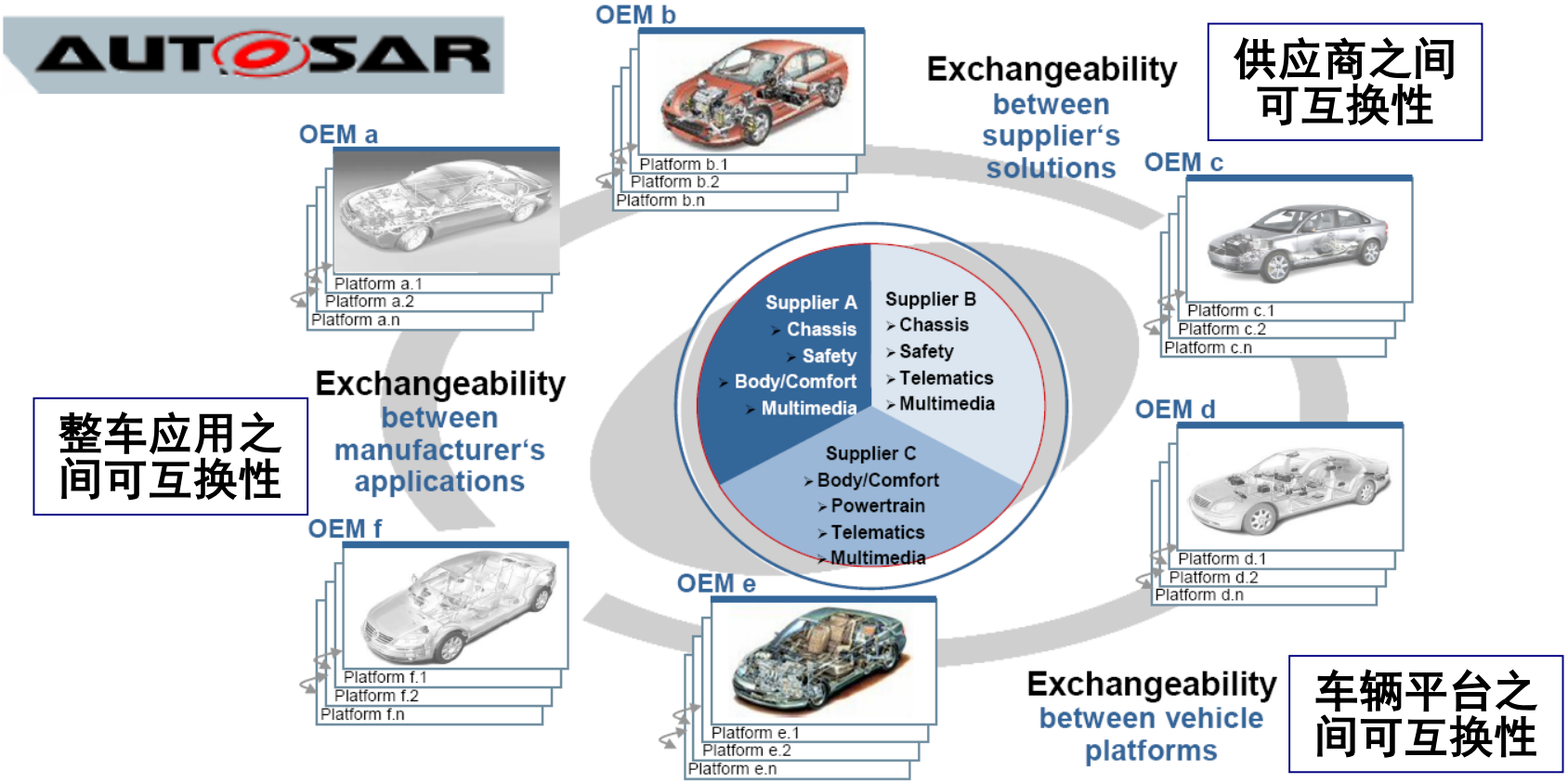
```
Temp1=X_High_Byte;
Temp2=X_Low_Byte;
X= Temp1*256+Temp2;
```

Task B读的结果为：
X=0x8033



§5.1 汽车电子控制系统的架构
Architecture of AECs

●AUTOSAR： AUTomotive Open System ARchitecture



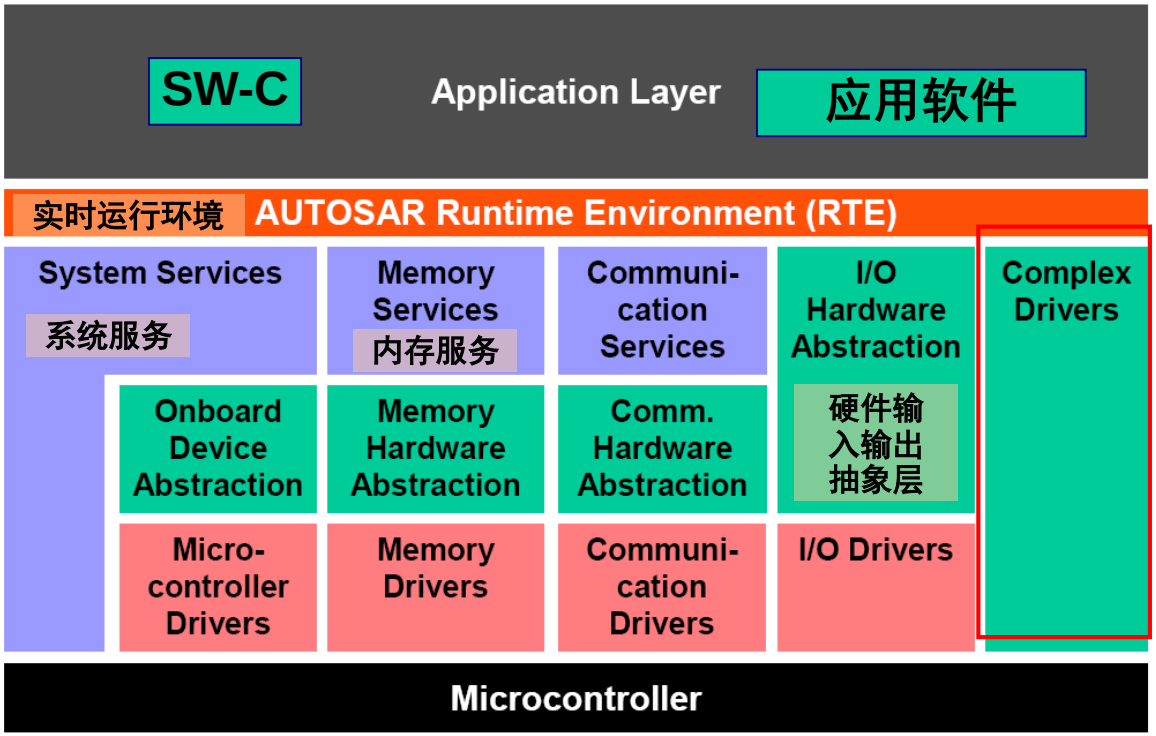


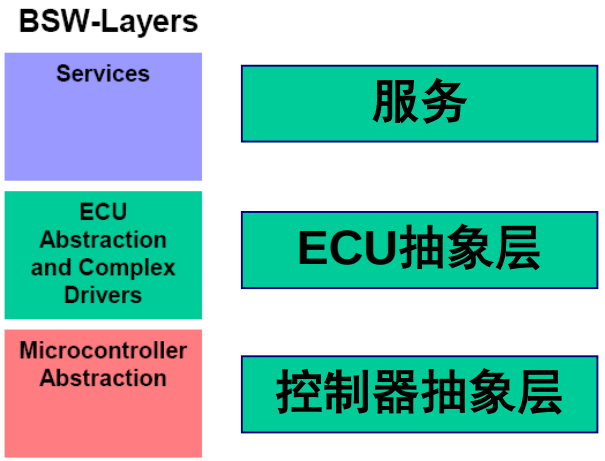
§5.1 汽车电子控制系统的架构
Architecture of AECs

●AUTOSAR 的层次/ Layers of AUTOSAR

- SW–C：Atomic SoftWare Component， cannot be distributed over several ECUs.
- RTE: Implements the VFB functionality on a specific ECU
- BSW: Basic SoftWare

复杂驱动（Complex Drivers，间称CD）指发动机控制中的曲轴同步，点火和喷射控制，以及电磁阀驱动等。







§5.1 汽车电子控制系统的架构
Architecture of AECs

- The SW–C is independent from the infrastructure:

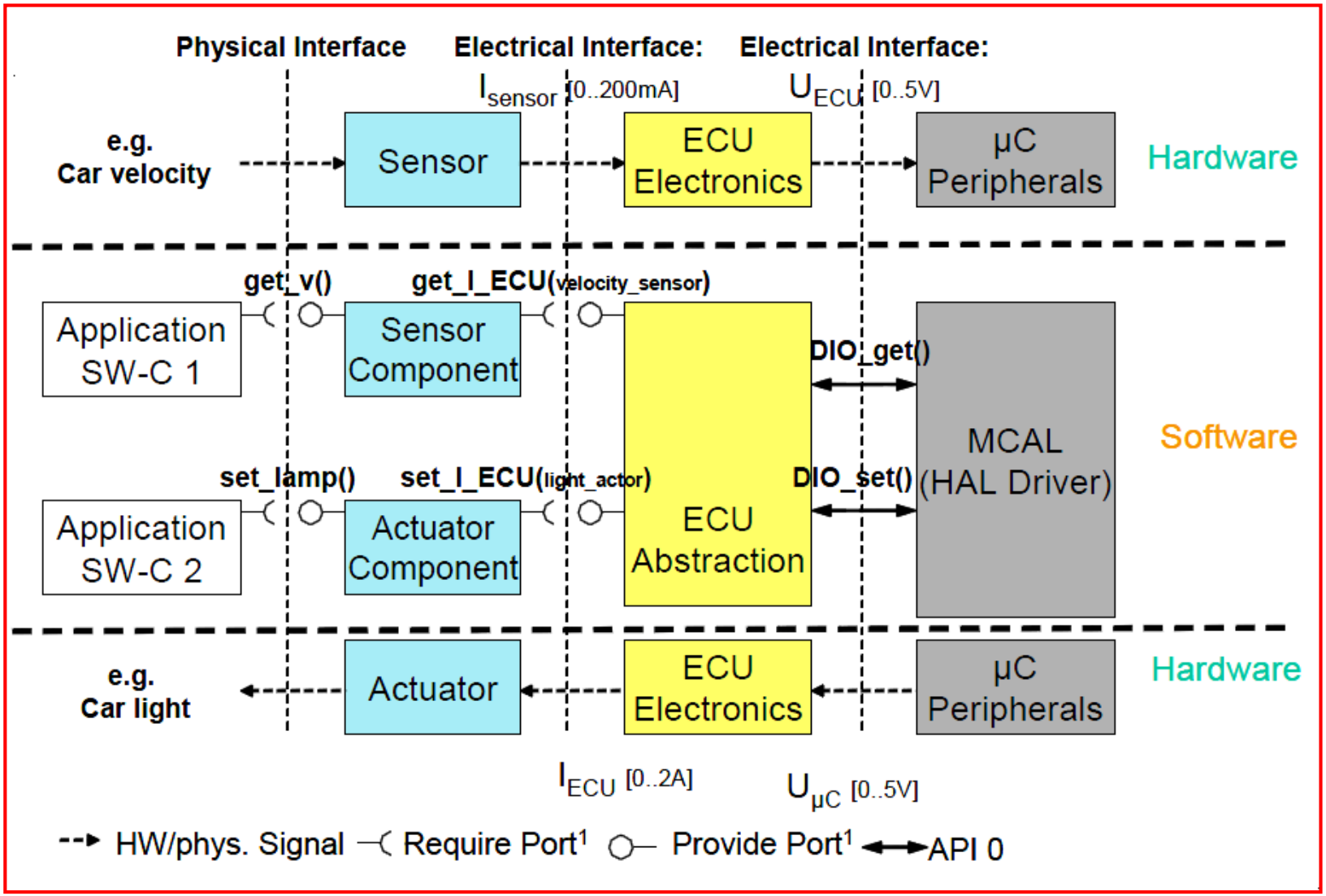
  /SW–C软件部件在源代码（Source Code）上能够独立于:

  - The Type of microcontroller of ECU on which the SW–C is Mapped/SW–C将要映射到的目标ECU中单片机的类型;
  - The type of ECU on which the SW–C is Mapped /SW–C将要映射到的目标ECU的类型;
  - The location of the other SW–C with which the SW–C interacts. /和本SW–C进行通信交互的其它SW–C所在的ECU位置，也就是不管进行通信交互的SW–C是否在同一个ECU中;
  - The Number of times a SW–C is instantiated in a system or within one ECU/本SW–C在同一个系统或ECU中实现次数的多少。



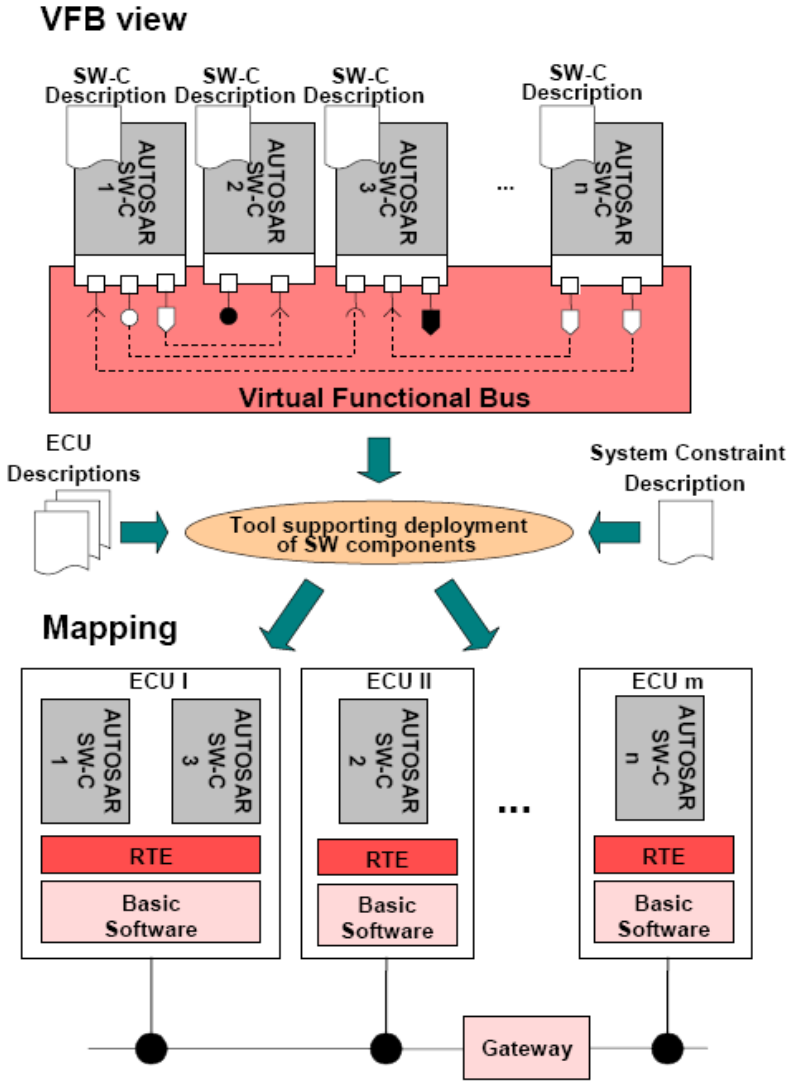
§5.1 汽车电子控制系统的架构
Architecture of AECs

The SW–C is independent from the infrastructure:





AUTOSAR的思路

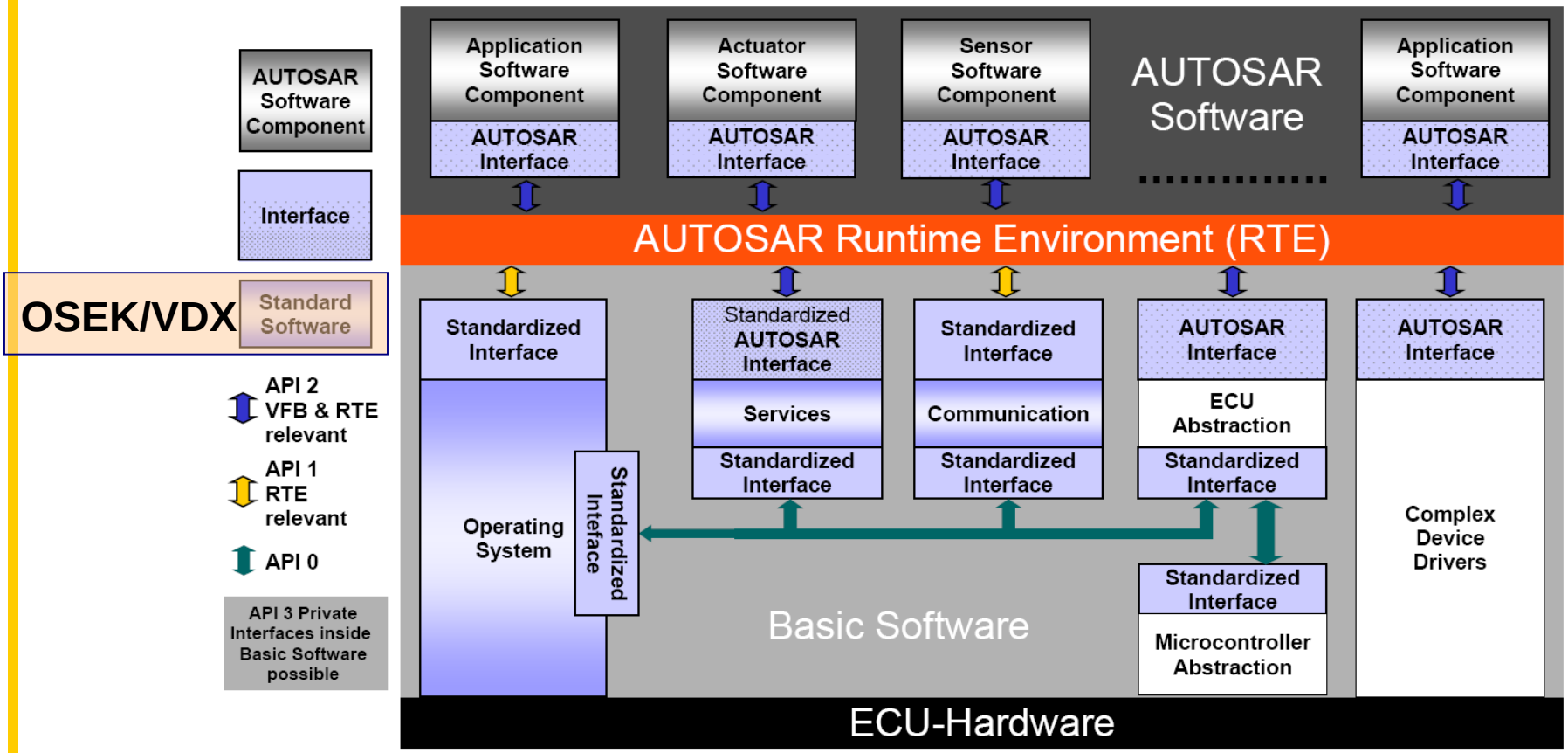


- 软件功能被定义一个个软件部件（Software Components，简称SW-C）；
- 以软件部件的说明为输入，在进行代码开发之前，先采用虚拟功能总线（Virtual Functional Bus，简称VFB）来验证各软件之间和对外接口的有效性；
- 将各个软件部件配置到各个ECU中，并配置各个ECU的基本软件模块（Basic Software，间称BSW）；
- 实时运行环境（Real Time Environment，简称RTE）负责SW-C和BSW之间的接口；
- SW-C与ECU无关，只有BSW才与ECU有关，软件功能独立于底层的软硬件，便于移植！



§5.1 汽车电子控制系统的架构
Architecture of AECs

●AUTOSAR的接口

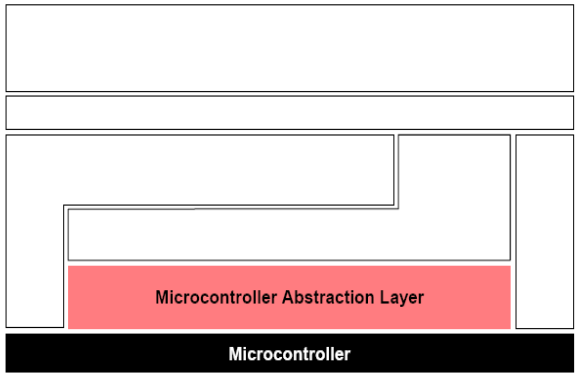


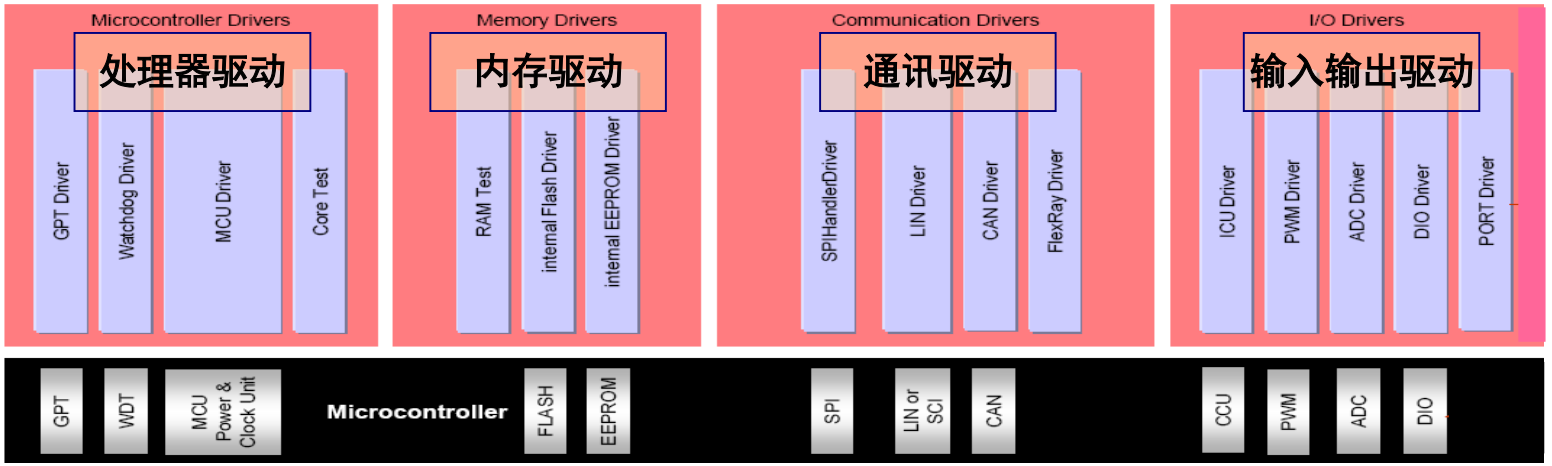


§5.1 汽车电子控制系统的架构
Architecture of AECs

●AUTOSAR：MAL

- ◆ 功能：是BSW的最底层，为微控制器（MCU）内部模块的驱动软件，直接访问MCU内部的各个外围模块，以及MCU外围存储器模块；
- ◆ 作用：使得上层的其它软件独立与控制器的硬件；
- ◆ 代码执行特点：与MCU直接相关；
- ◆上层接口：与MCU独立的标准化接口
- ◆分为：处理器驱动，存储器驱动，通讯驱动和输入输出驱动

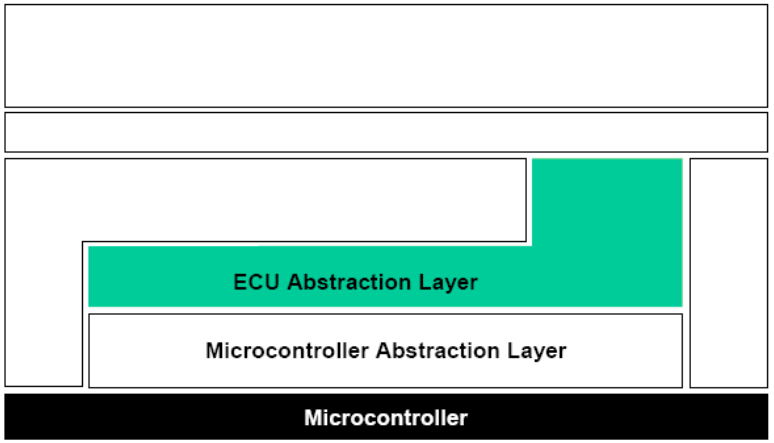






§5.1 汽车电子控制系统的架构
Architecture of AECs

- AUTOSAR：ECU AL
  - ECU抽象层与控制器抽象层接口，也包括了MCU外部设备的驱动；
  - ECU抽象层通过API接口访问MCU的内部模块和外围模块，而不用考虑模块的物理地址（MCU的内部还是外部），也不用考虑模块与MCU的接口方式（并口，SPI，串口等）；
  - 功能：ECU AL的目标是更上层软件独立于ECU的硬件；
  - 代码执行特点：与MCU无关，但与ECU有关；
  - 上层接口：与MCU和ECU硬件都无关，只有信号的类型有关。
  - 包括：
    - 通讯硬件抽象层，COM HW
    - 输入输出硬件抽象层，I/O HW
    - 存储器硬件抽象层，Memory HW
    - 片上设备硬件抽象层，Onboard HW



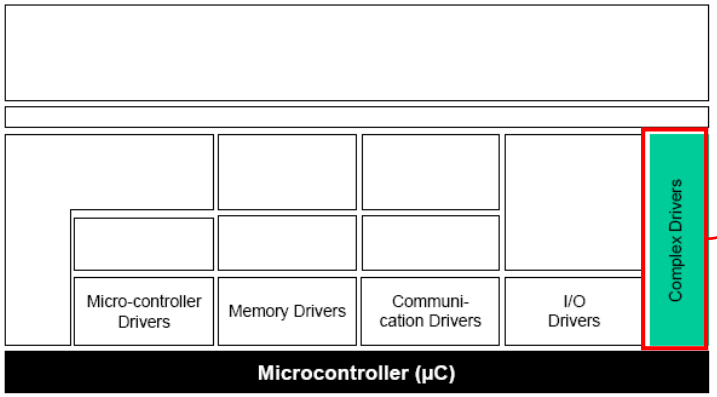


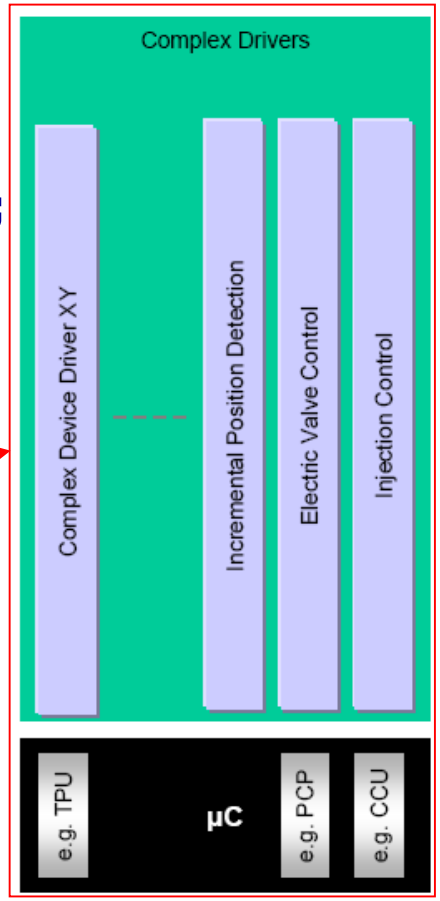
§5.1 汽车电子控制系统的架构
Architecture of AECs

●AUTOSAR：CD

- ◆ 复杂驱动（Complex Drivers，简称CD）
- ◆ CD主要执行检测复杂的传感器和控制执行器，通过直接访问微处理器的特殊中断或其内部的复杂周边模块，例如TPU，PCP等；主要用于喷射控制，电磁阀控制，曲轴位置检测等，
- ◆功能：接口传感器和执行器完成特定功能和定时要求；
- ◆ 代码执行特点：和MCU、ECU及应用程序都高度无关；
- ◆ 上层接口：根据AUTOSAR接口定义和执行。

CD 的技术标准并没有给出

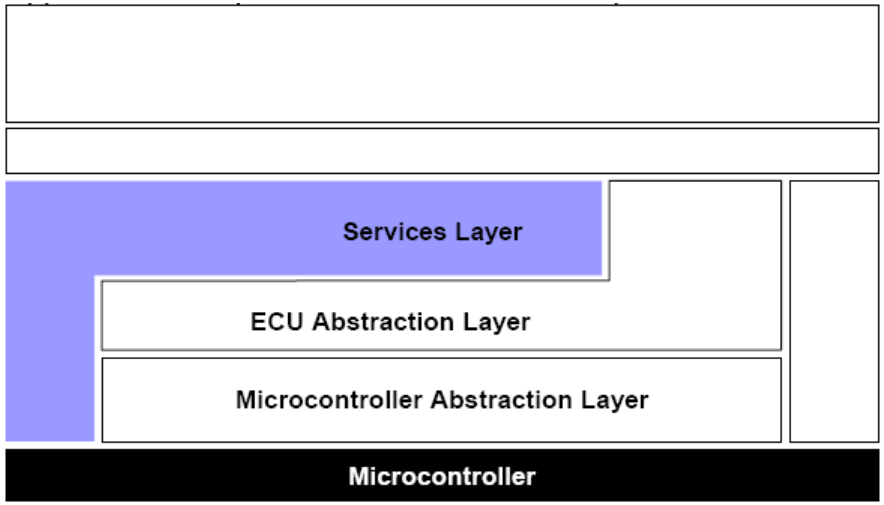






§5.1 汽车电子控制系统的架构
Architecture of AECs

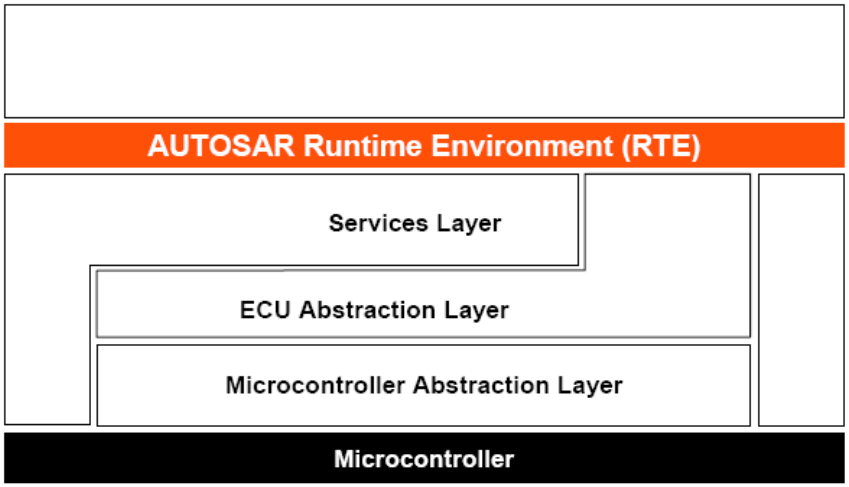
- AUTOSAR：SL
  - 是基本软件BSW的最上层，通过ECU的抽象层访问底层的I/O信号；
  - SL包括了：操作系统功能，车辆网络通讯和管理服务， 存储器服务，诊断服务，ECU的状态管理；
  - 功能：为上层应用提供基本服务和基本的软件模块；
  - 代码执行特点：部分和单片机、ECU和应用系统相关；
  - 上层接口：与单片机和ECU都无关。





●AUTOSAR：RTE

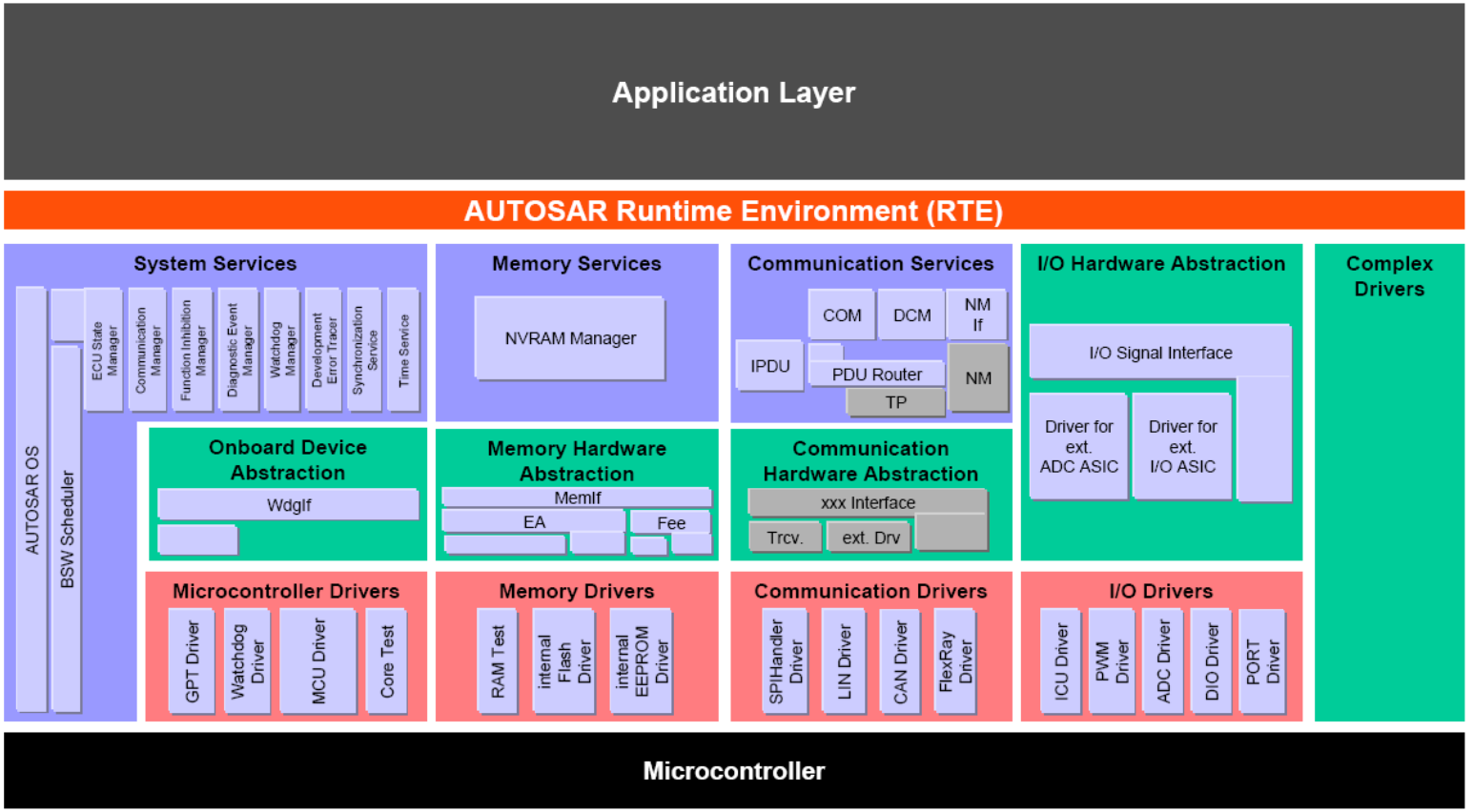
- ◆ RTE是专门为上层应用程序（包括软件部件、传感器部件和执行器部件）提供通讯服务的实时运行环境；
- ◆ 在RTE的上层软件架构就从层（Layered）转为（Component style）；AUTOSAR的软件部件（SW-C）通过RTE服务与ECU内部或者外部的软件部件（SW-C）建立通讯；
- ◆ 功能：使得AUTOSAR软件部件独立于所处的ECU。
- ◆ 代码执行特点：由ECU和应用指定（一般和所在ECU相关）；
- ◆ 上层接口：完全独立所处的ECU，基于ECU无关；





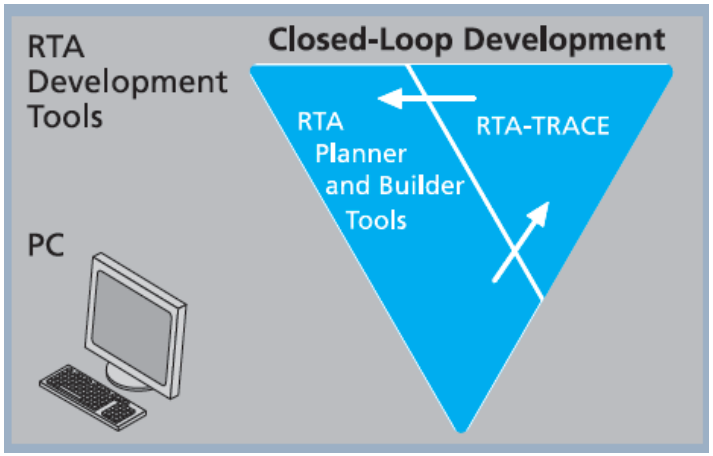
§5.1 汽车电子控制系统的架构
Architecture of AECs

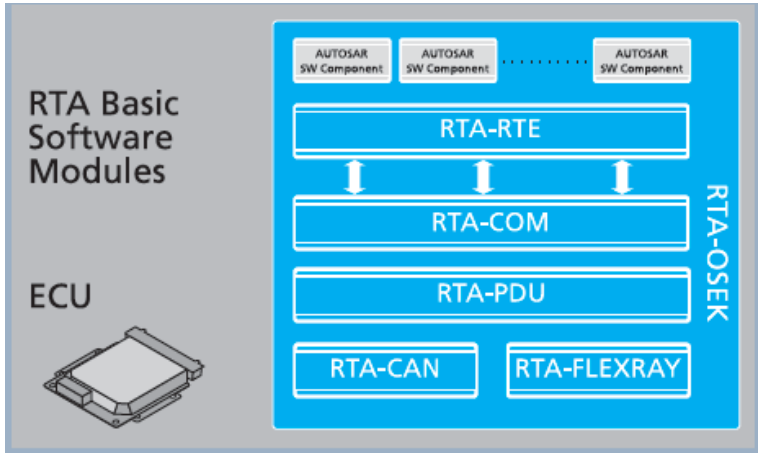
- AUTOSAR：ICC
  - Implementation Conformance Classis





- 满足AUTOSAR的开发工具
  - RTA – Real-Time AUTOSAR
    - RTA-OSEK
    - RTA-RTE
    - RTA-COM
    - RTA-PDU
    - RTA-CAN







HarmonyOS 鸿蒙系统 | 鸿蒙学堂 (hmxt.org)